

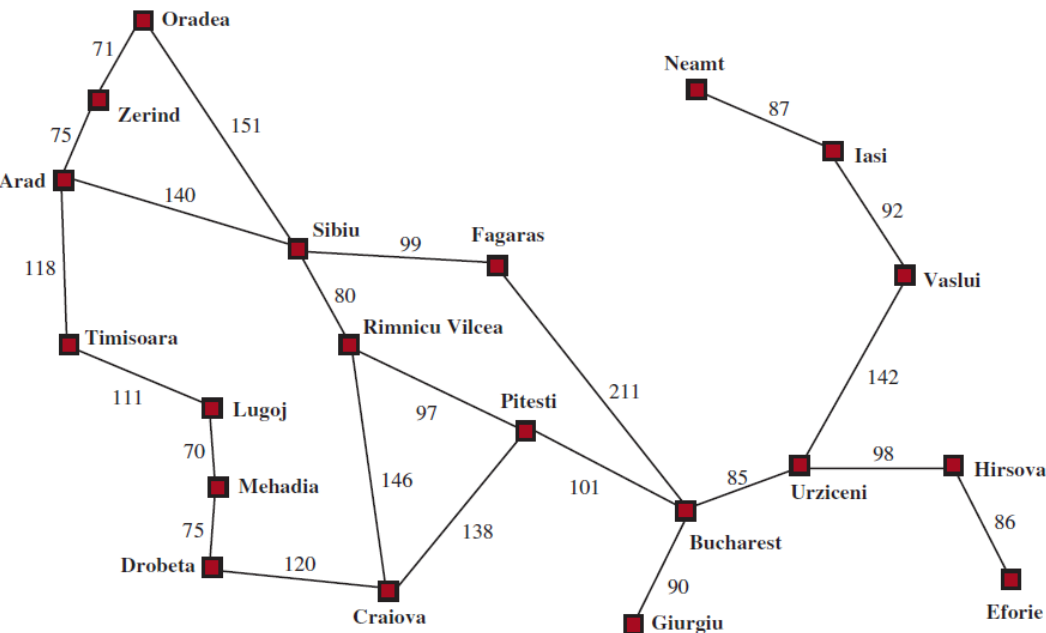
Continuous Space & Nondeterministic Actions

Outline

- I. Local search in continuous spaces
- II. Search with non-deterministic actions

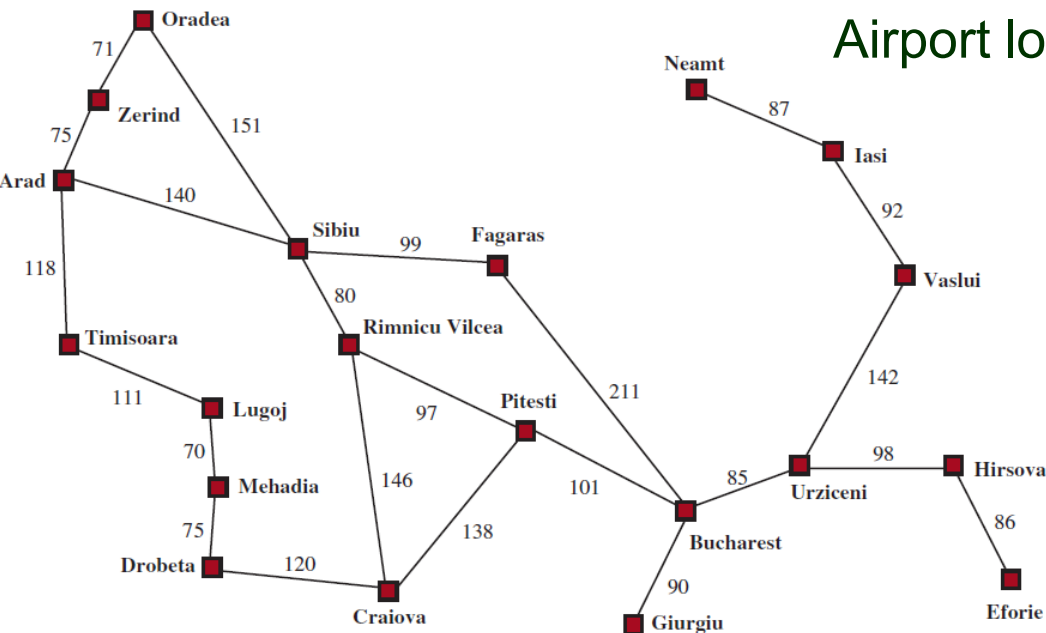
I. Local Search in Continuous Space

Problem Place three new airports anywhere in Romania to minimize the sum of squared straight-line distances from every city to its nearest airport.



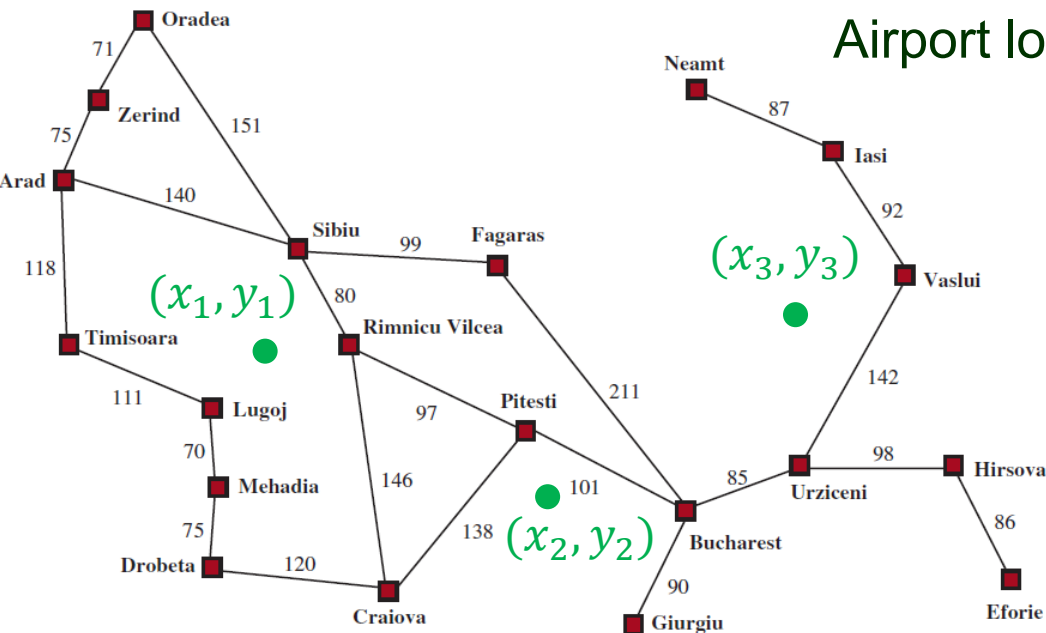
I. Local Search in Continuous Space

Problem Place three new airports anywhere in Romania to minimize the sum of squared straight-line distances from every city to its nearest airport.



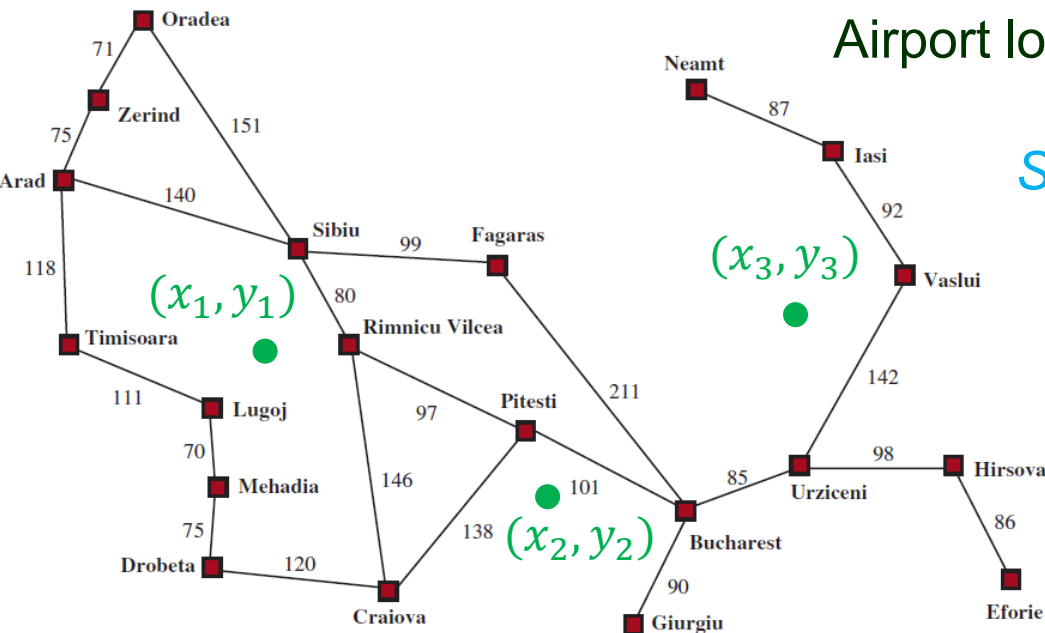
I. Local Search in Continuous Space

Problem Place three new airports anywhere in Romania to minimize the sum of squared straight-line distances from every city to its nearest airport.



I. Local Search in Continuous Space

Problem Place three new airports anywhere in Romania to minimize the sum of squared straight-line distances from every city to its nearest airport.

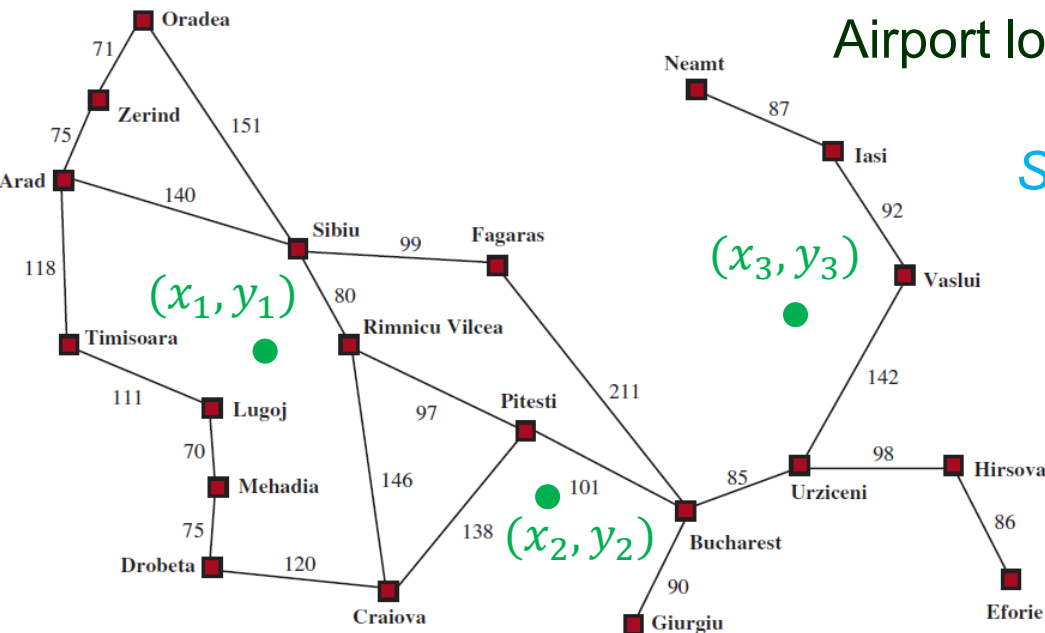


Airport locations: $(x_1, y_1), (x_2, y_2), (x_3, y_3)$

State $\mathbf{x} = (x_1, y_1, x_2, y_2, x_3, y_3)$

I. Local Search in Continuous Space

Problem Place three new airports anywhere in Romania to minimize the sum of squared straight-line distances from every city to its nearest airport.



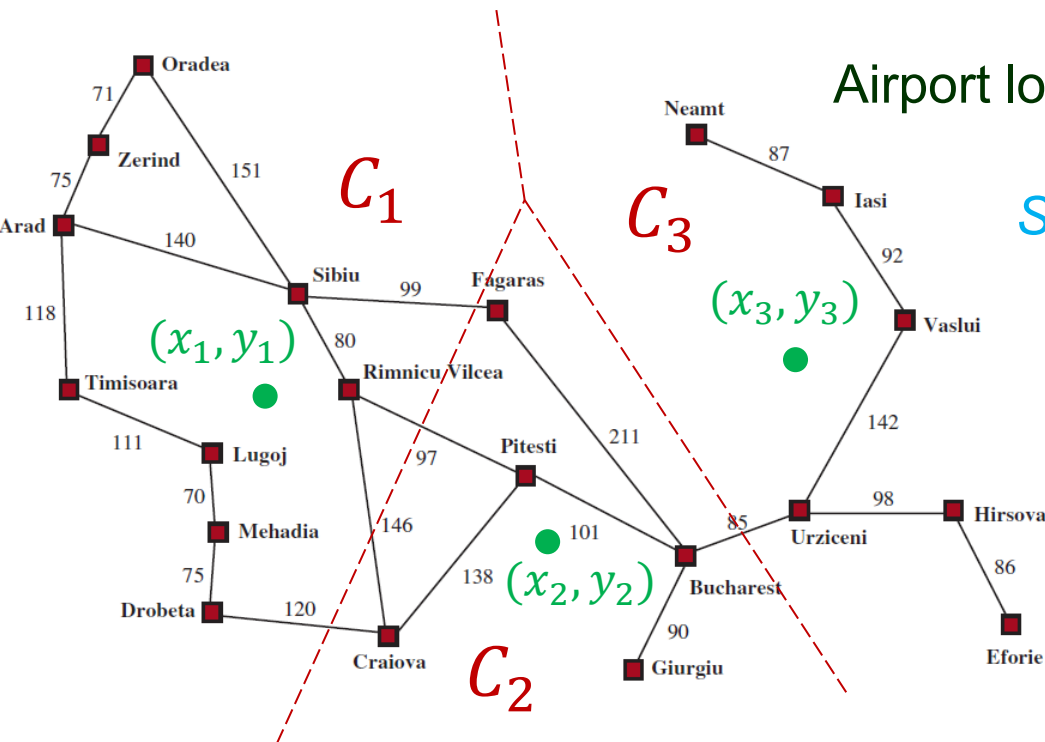
Airport locations: $(x_1, y_1), (x_2, y_2), (x_3, y_3)$

State $\mathbf{x} = (x_1, y_1, x_2, y_2, x_3, y_3)$

C_i : set of cities to which airport i is the closest, for $1 \leq i \leq 3$.
Depends on $\mathbf{x}$.

I. Local Search in Continuous Space

Problem Place three new airports anywhere in Romania to minimize the sum of squared straight-line distances from every city to its nearest airport.



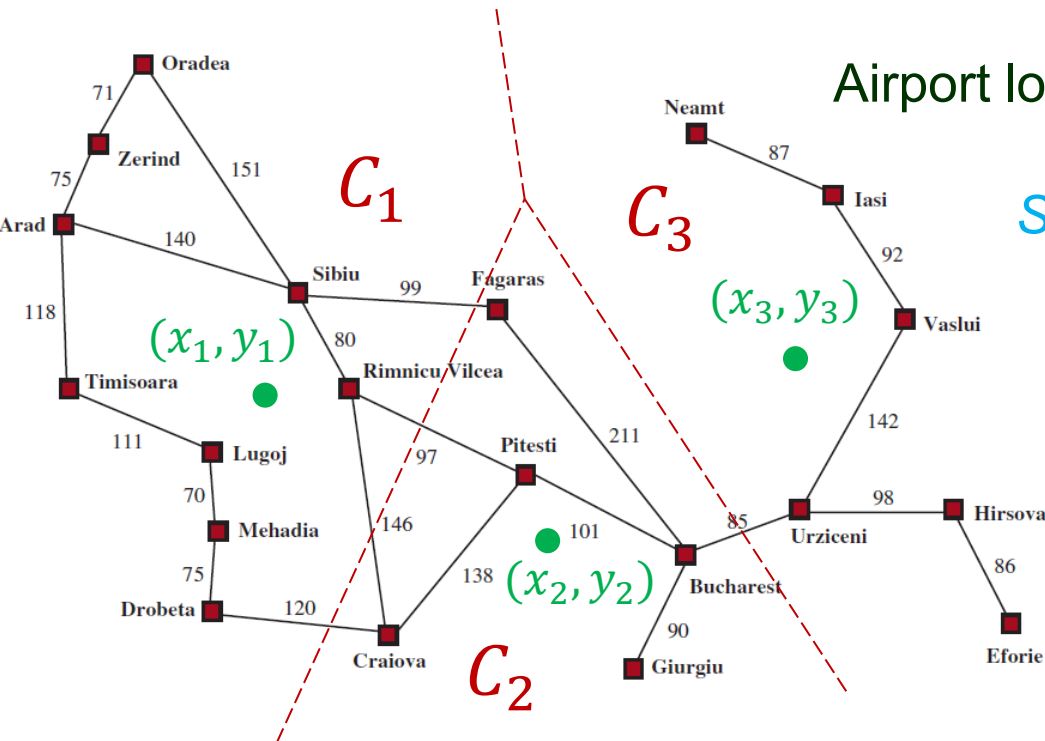
Airport locations: (x_1, y_1) , (x_2, y_2) , (x_3, y_3)

State $\mathbf{x} = (x_1, y_1, x_2, y_2, x_3, y_3)$

C_i : set of cities to which airport i is the closest, for $1 \leq i \leq 3$.
Depends on $\mathbf{x}$.

I. Local Search in Continuous Space

Problem Place three new airports anywhere in Romania to minimize the sum of squared straight-line distances from every city to its nearest airport.



Airport locations: (x_1, y_1) , (x_2, y_2) , (x_3, y_3)

State $\mathbf{x} = (x_1, y_1, x_2, y_2, x_3, y_3)$

C_i : set of cities to which airport i is the closest, for $1 \leq i \leq 3$.
Depends on $\mathbf{x}$.

Objective function:

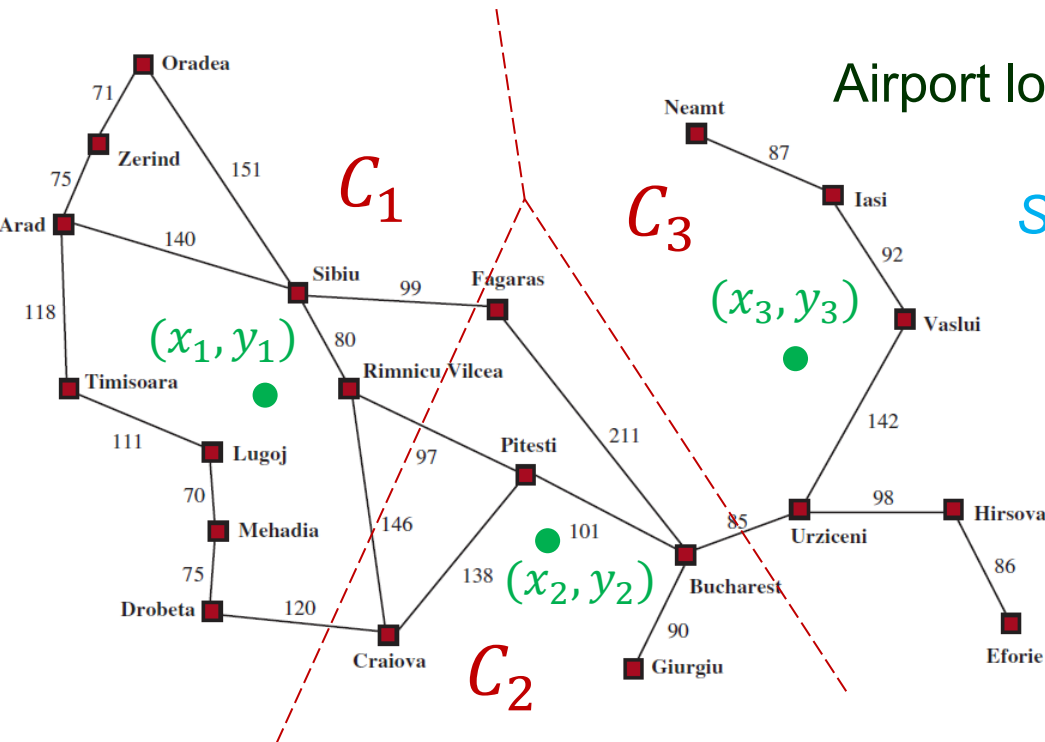
$$f(\mathbf{x}) = f(x_1, y_1, x_2, y_2, x_3, y_3)$$

$$= \sum_{i=1}^3 \sum_{c \in C_i(\mathbf{x})} ((x_i - x_c)^2 + (y_i - y_c)^2)$$

Voronoi diagram
(COMS 4180/5180,
Spring 2026)

I. Local Search in Continuous Space

Problem Place three new airports anywhere in Romania to minimize the sum of squared straight-line distances from every city to its nearest airport.



Airport locations: $(x_1, y_1), (x_2, y_2), (x_3, y_3)$

State $\mathbf{x} = (x_1, y_1, x_2, y_2, x_3, y_3)$

C_i : set of cities to which airport i is the closest, for $1 \leq i \leq 3$.
Depends on $\mathbf{x}$.

Objective function:

$$f(\mathbf{x}) = f(x_1, y_1, x_2, y_2, x_3, y_3)$$

$$= \sum_{i=1}^3 \sum_{c \in C_i(\mathbf{x})} ((x_i - x_c)^2 + (y_i - y_c)^2)$$

Voronoi diagram
(COMS 4180/5180,
Spring 2026)

$$\min_{\mathbf{x}} f(\mathbf{x})$$

Gradient-Based Method

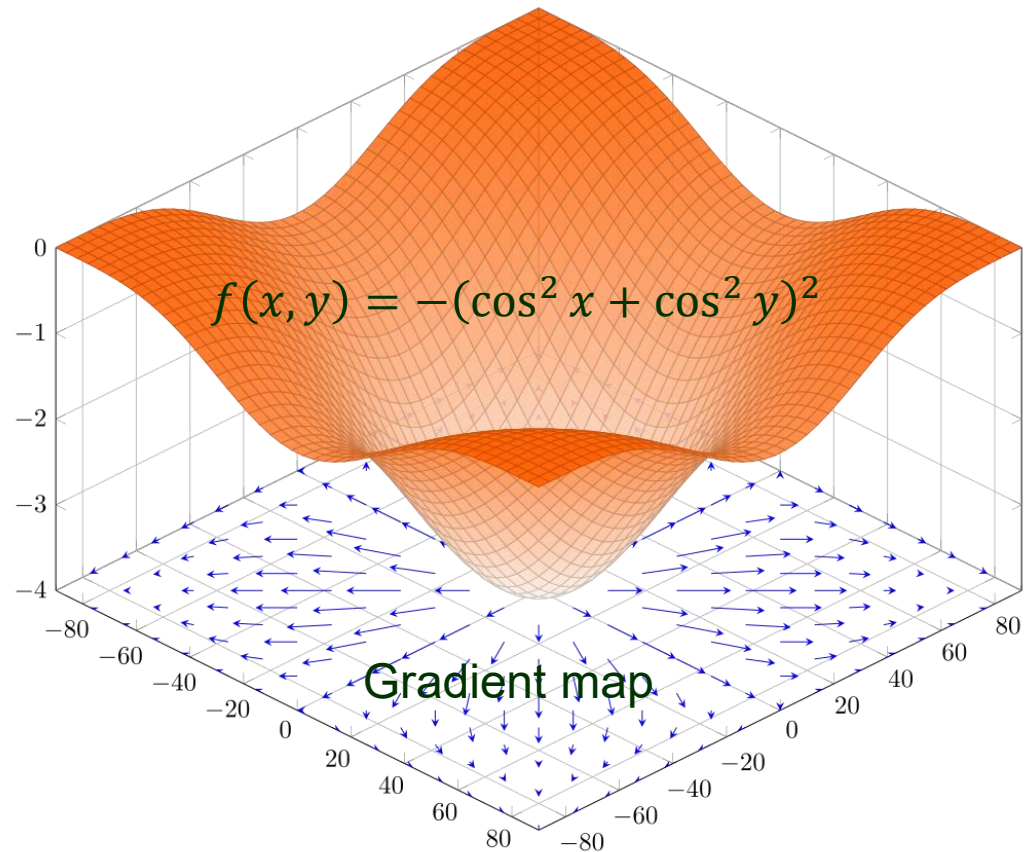
Use the *gradient*: $\nabla f(x_1, \dots, x_n) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$

Gradient-Based Method

Use the *gradient*: $\nabla f(x_1, \dots, x_n) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$
 direction of fastest increase in f value locally

Gradient-Based Method

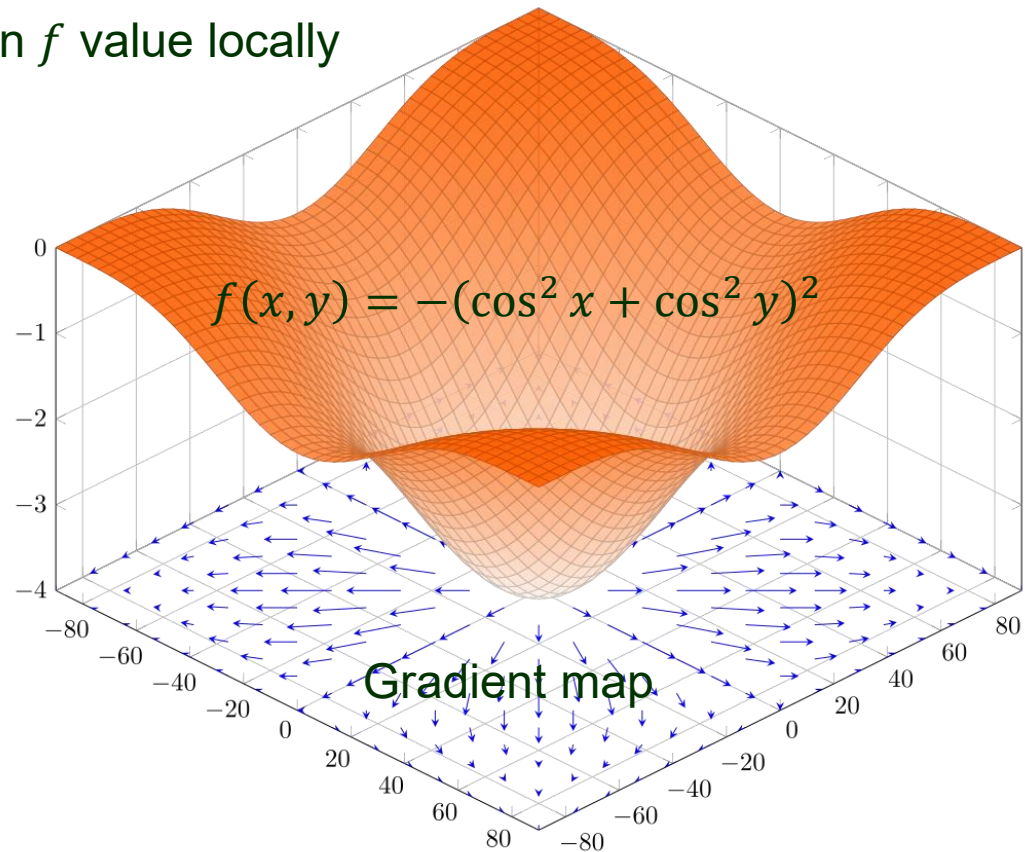
Use the *gradient*: $\nabla f(x_1, \dots, x_n) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$
↑
direction of *fastest* increase in f value locally



Gradient-Based Method

Use the *gradient*: $\nabla f(x_1, \dots, x_n) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$
↑
direction of *fastest* increase in f value locally

$-\nabla f$: direction of *fastest* decrease in f value locally



Gradient Descent

Back to airport placements:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3} \right)$$

$$f(\mathbf{x}) = \sum_{i=1}^3 \sum_{c \in C_i(\mathbf{x})} ((x_i - x_c)^2 + (y_i - y_c)^2)$$

Gradient Descent

Back to airport placements:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3} \right)$$

$$f(\mathbf{x}) = \sum_{i=1}^3 \sum_{c \in C_i(\mathbf{x})} ((x_i - x_c)^2 + (y_i - y_c)^2) \implies \frac{\partial f}{\partial x_1} = 2 \sum_{c \in C_1} (x_1 - x_c)$$

Gradient Descent

Back to airport placements:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3} \right)$$

$$f(\mathbf{x}) = \sum_{i=1}^3 \sum_{c \in C_i(\mathbf{x})} ((x_i - x_c)^2 + (y_i - y_c)^2) \implies \frac{\partial f}{\partial x_1} = 2 \sum_{c \in C_1} (x_1 - x_c)$$

Update: $\mathbf{x} \leftarrow \mathbf{x} - \alpha \nabla f(\mathbf{x})$

Gradient Descent

Back to airport placements:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3} \right)$$

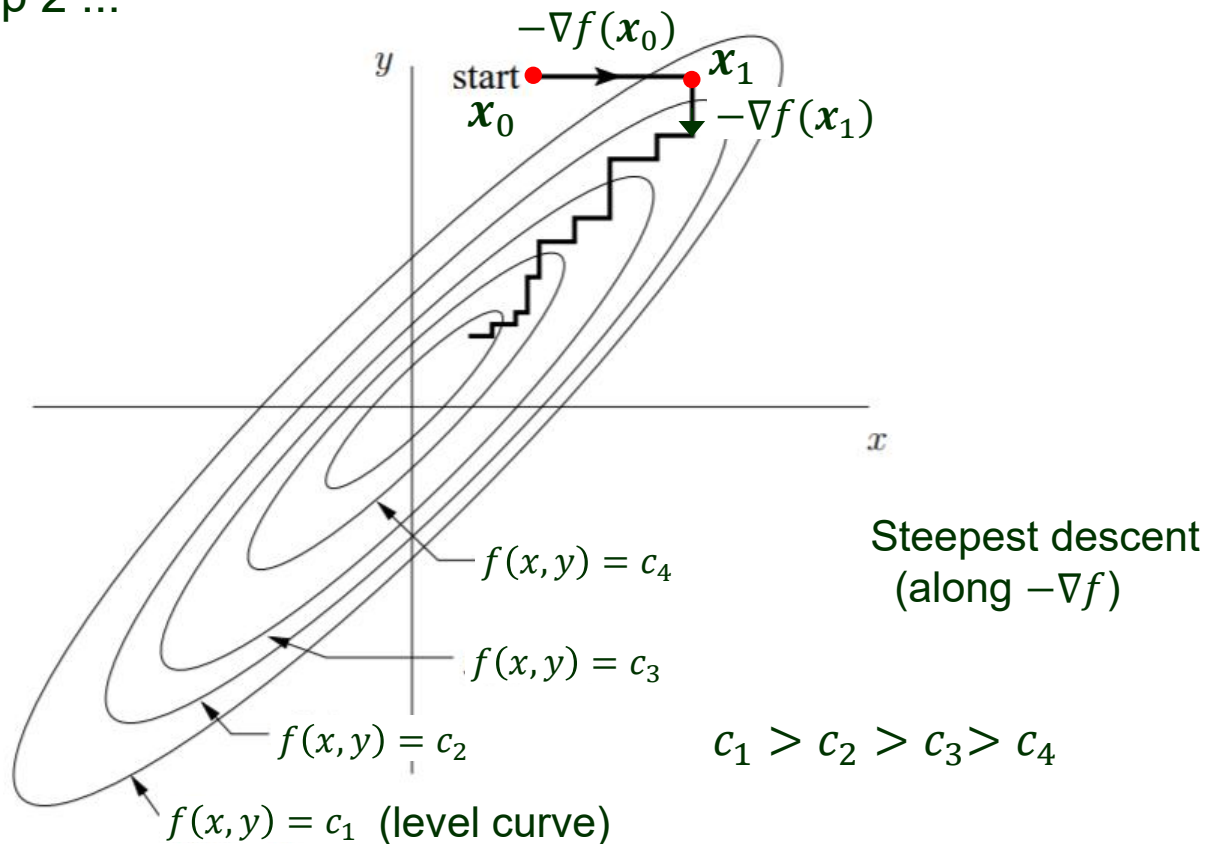
$$f(\mathbf{x}) = \sum_{i=1}^3 \sum_{c \in C_i(\mathbf{x})} ((x_i - x_c)^2 + (y_i - y_c)^2) \implies \frac{\partial f}{\partial x_1} = 2 \sum_{c \in C_1} (x_1 - x_c)$$

Update: $\mathbf{x} \leftarrow \mathbf{x} - \alpha \nabla f(\mathbf{x})$

↑
step size

Line Search

1. Start at an initial state $x = x_0$.
2. Move along $-\nabla f(x)$ until f no longer decreases.
3. $x \leftarrow$ new stopping point.
4. Go back to step 2 ...



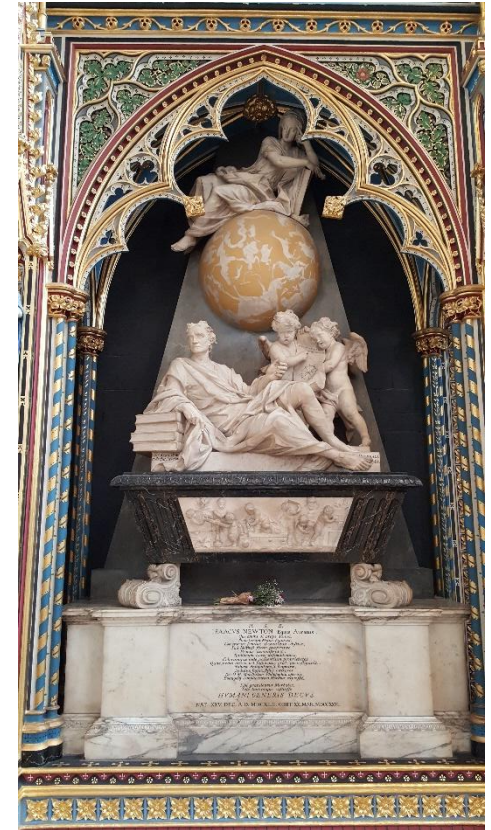
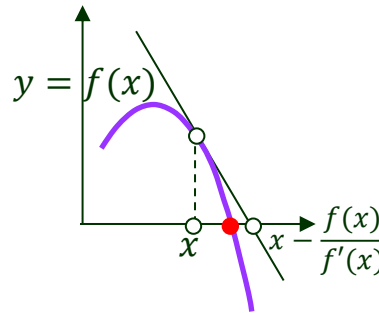
Newton-Raphson Method

- Solve

$$f(x) = 0 \quad // \text{ one variable}$$

with the iteration formula

$$x \leftarrow x - \frac{f(x)}{f'(x)}$$



Newton's monument (& sarcophagus)
Westminster Abbey, London, UK

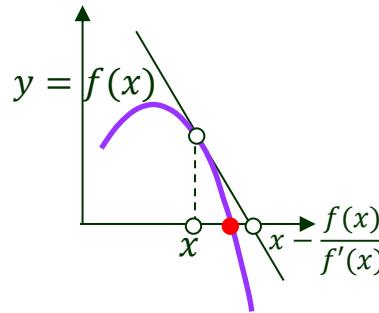
Newton-Raphson Method

- Solve

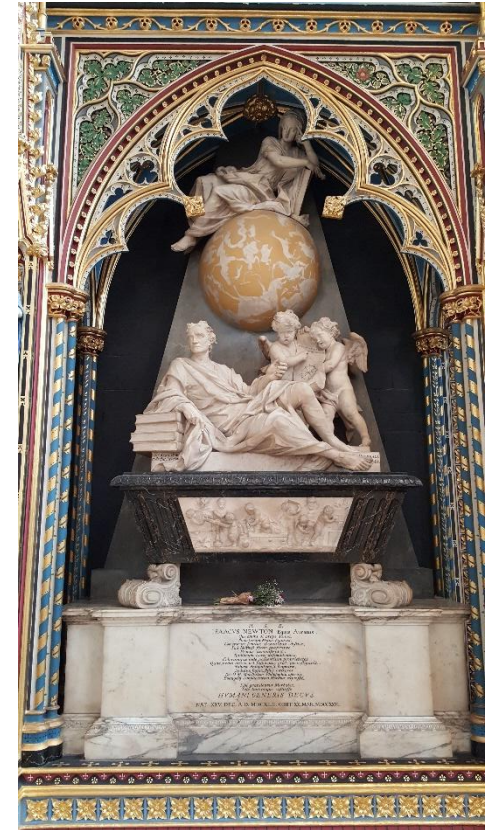
$$f(x) = 0 \quad // \text{ one variable}$$

with the iteration formula

$$x \leftarrow x - \frac{f(x)}{f'(x)}$$



- To maximize/minimize $f(x)$, find x at which $\nabla f(x) = 0$.



[Newton's monument \(& sarcophagus\)](#)
[Westminster Abbey, London, UK](#)

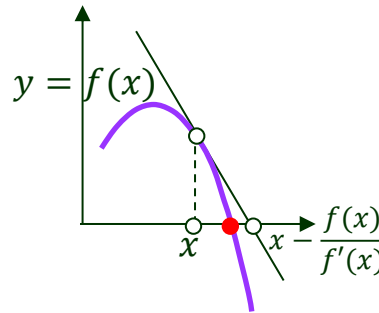
Newton-Raphson Method

- Solve

$$f(x) = 0 \quad // \text{ one variable}$$

with the iteration formula

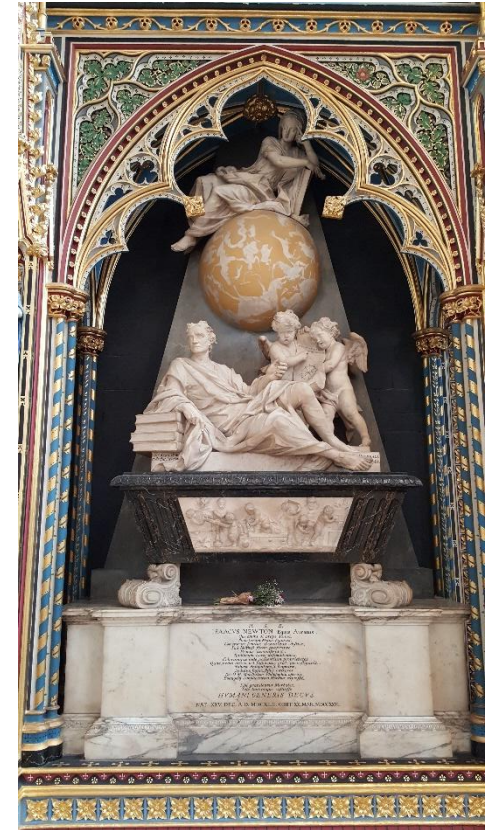
$$x \leftarrow x - \frac{f(x)}{f'(x)}$$



- To maximize/minimize $f(x)$, find x at which $\nabla f(x) = \mathbf{0}$.

Iteration formula:

$$x \leftarrow x - H_f^{-1}(x)(\nabla f(x))^T$$



[Newton's monument \(& sarcophagus\)](#)
[Westminster Abbey, London, UK](#)

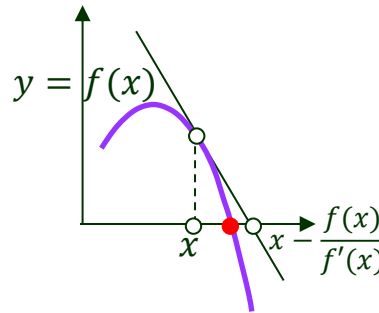
Newton-Raphson Method

- Solve

$$f(x) = 0 \quad // \text{ one variable}$$

with the iteration formula

$$x \leftarrow x - \frac{f(x)}{f'(x)}$$

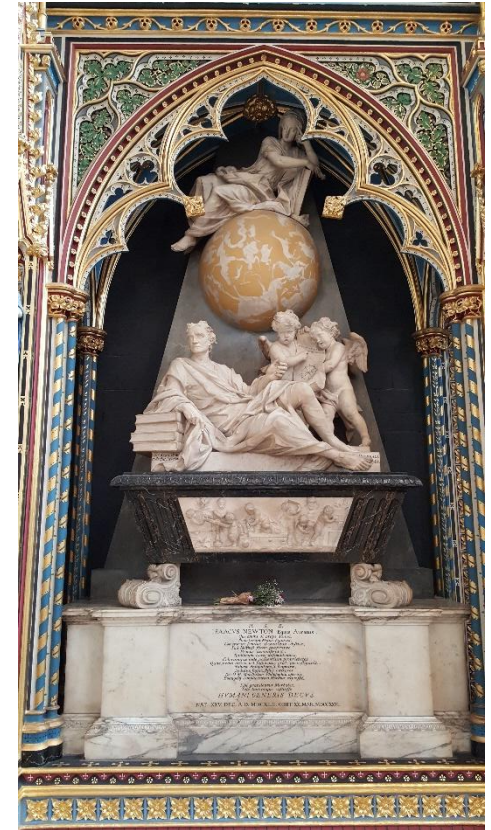


- To maximize/minimize $f(x)$, find x at which $\nabla f(x) = \mathbf{0}$.

Iteration formula:

$$x \leftarrow x - H_f^{-1}(x)(\nabla f(x))^T$$

Hessian H_f of f : matrix $\left(\frac{\partial^2 f}{\partial x_i \partial x_j} \right)$



[Newton's monument \(& sarcophagus\)](#)
[Westminster Abbey, London, UK](#)

More on Continuous Optimization

Com S 4770/5770 (Fall 2026) covers more topics:

- ◆ Unconstrained nonlinear optimization:

<https://faculty.sites.iastate.edu/jia/files/inline-files/nonlinear-program.pdf>

More on Continuous Optimization

Com S 4770/5770 (Fall 2026) covers more topics:

- ◆ Unconstrained nonlinear optimization:

<https://faculty.sites.iastate.edu/jia/files/inline-files/nonlinear-program.pdf>

- ◆ Constrained nonlinear optimization (Lagrange multipliers):

<https://faculty.sites.iastate.edu/jia/files/inline-files/lagrange-multiplier.pdf>

More on Continuous Optimization

Com S 4770/5770 (Fall 2026) covers more topics:

- ◆ Unconstrained nonlinear optimization:

<https://faculty.sites.iastate.edu/jia/files/inline-files/nonlinear-program.pdf>

- ◆ Constrained nonlinear optimization (Lagrange multipliers):

<https://faculty.sites.iastate.edu/jia/files/inline-files/lagrange-multiplier.pdf>

- ◆ Linear programming (i.e., linear optimization under linear constraints)

<https://faculty.sites.iastate.edu/jia/files/inline-files/linear-program.pdf>

<https://faculty.sites.iastate.edu/jia/files/inline-files/simplex.pdf>

II. Nondeterministic Actions

The agent **may not know the next state** after taking an action in a nondeterministic environment.

II. Nondeterministic Actions

The agent **may not know the next state** after taking an action in a nondeterministic environment.

A ***belief state*** is a set of physical states believed to be possible by the agent.

II. Nondeterministic Actions

The agent **may not know the next state** after taking an action in a nondeterministic environment.

A **belief state** is a set of physical states believed to be possible by the agent.

Problem solution: a **conditional plan**.

II. Nondeterministic Actions

The agent **may not know the next state** after taking an action in a nondeterministic environment.

A **belief state** is a set of physical states believed to be possible by the agent.

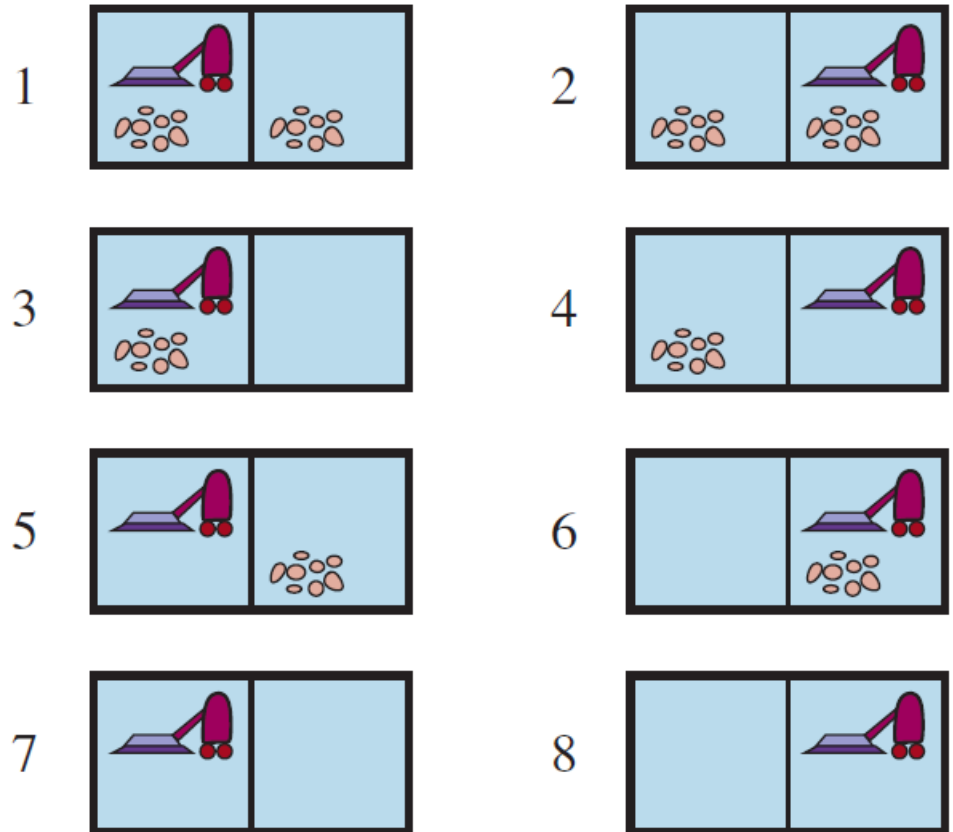
Problem solution: a **conditional plan**.



To specify what to do depending on what percepts the agent receives while executing the plan.

Perfect Vacuum World

Fully observable, deterministic, and completely known



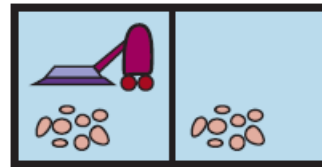
8 possible states

Perfect Vacuum World

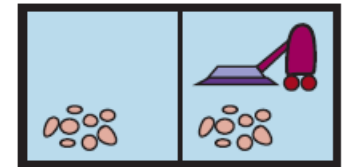
Fully observable, deterministic, and completely known

Solution: *Suck, Right, Suck* ←

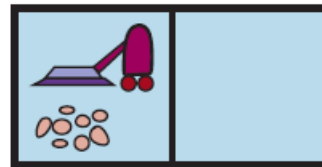
1



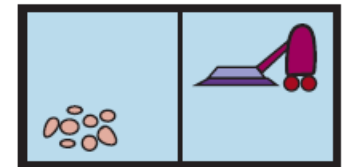
2



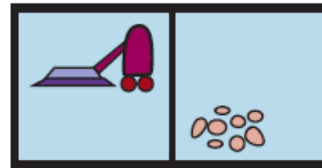
3



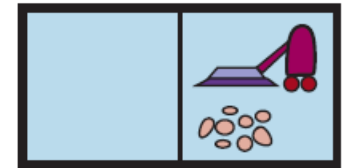
4



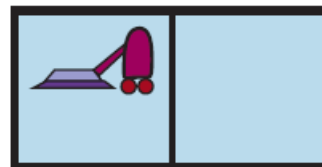
5



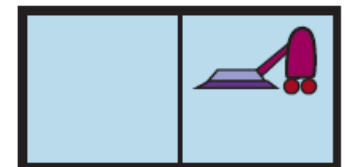
6



7



8

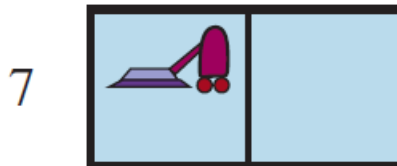
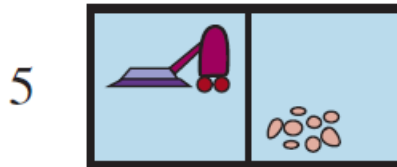
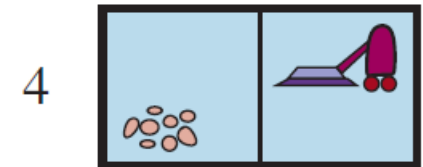
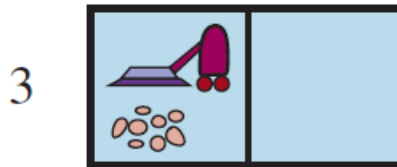
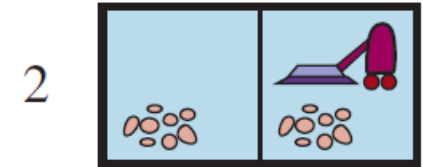
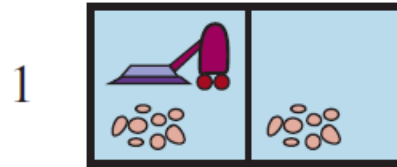


8 possible states

Erratic Vacuum World

Suck:

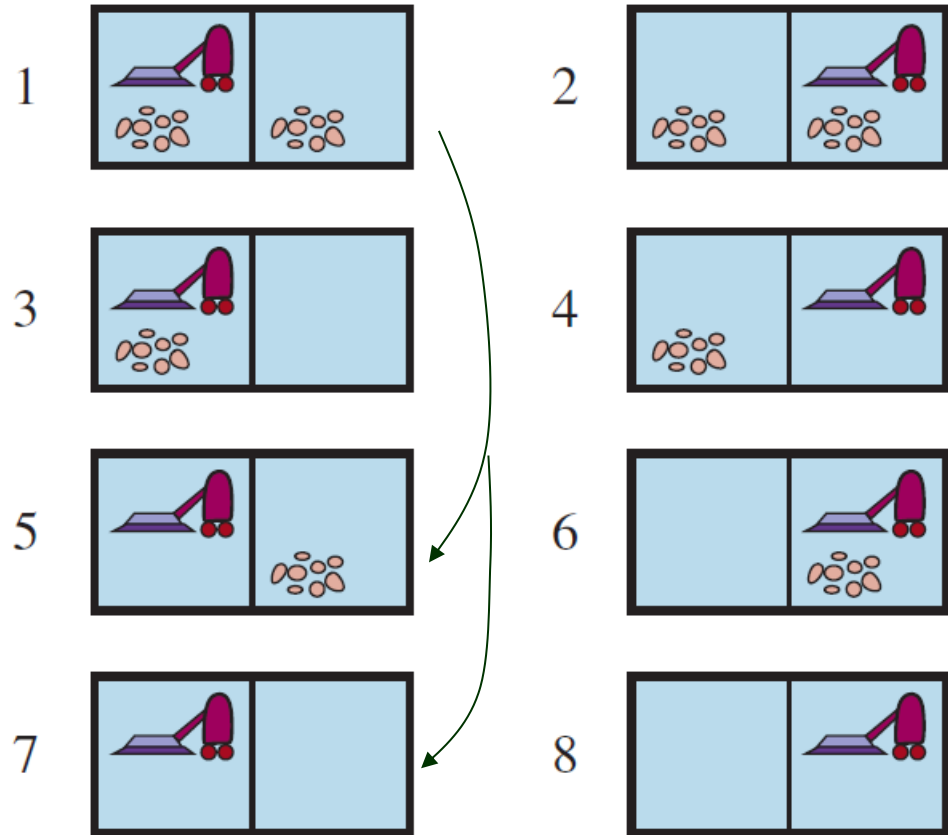
- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square



Erratic Vacuum World

Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

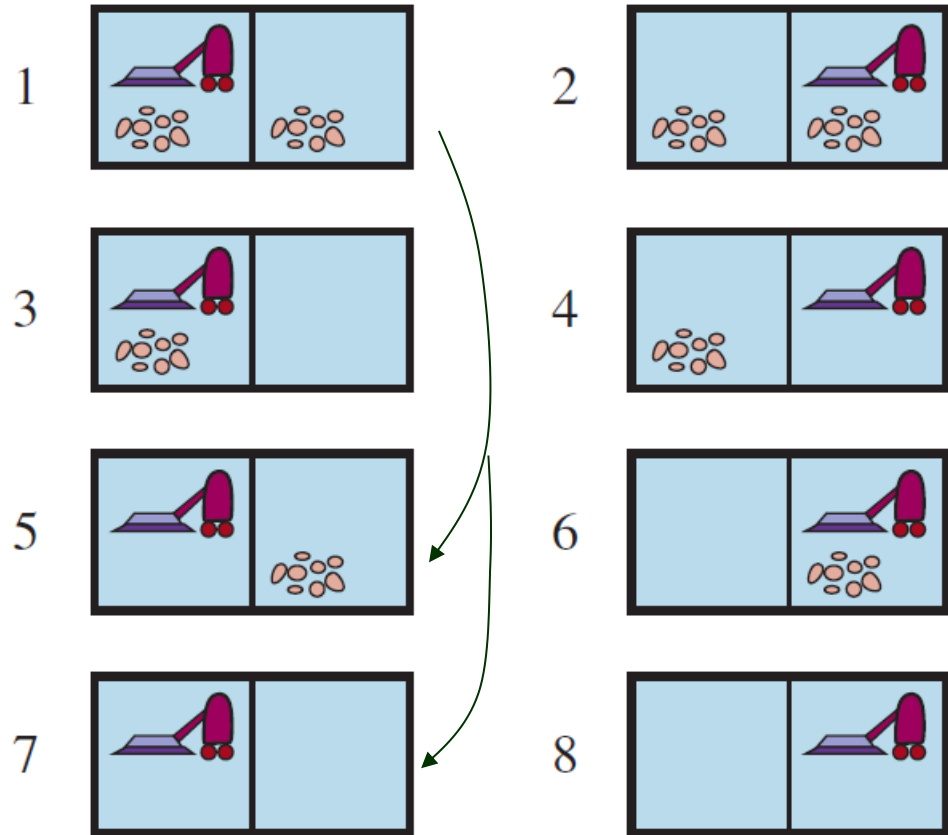


Either state 5 or 7 after applied to state 1:

Erratic Vacuum World

Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square



Either state 5 or 7 after applied to state 1:

$$\text{RESULTS}(1, \text{Suck}) = \{5, 7\}$$

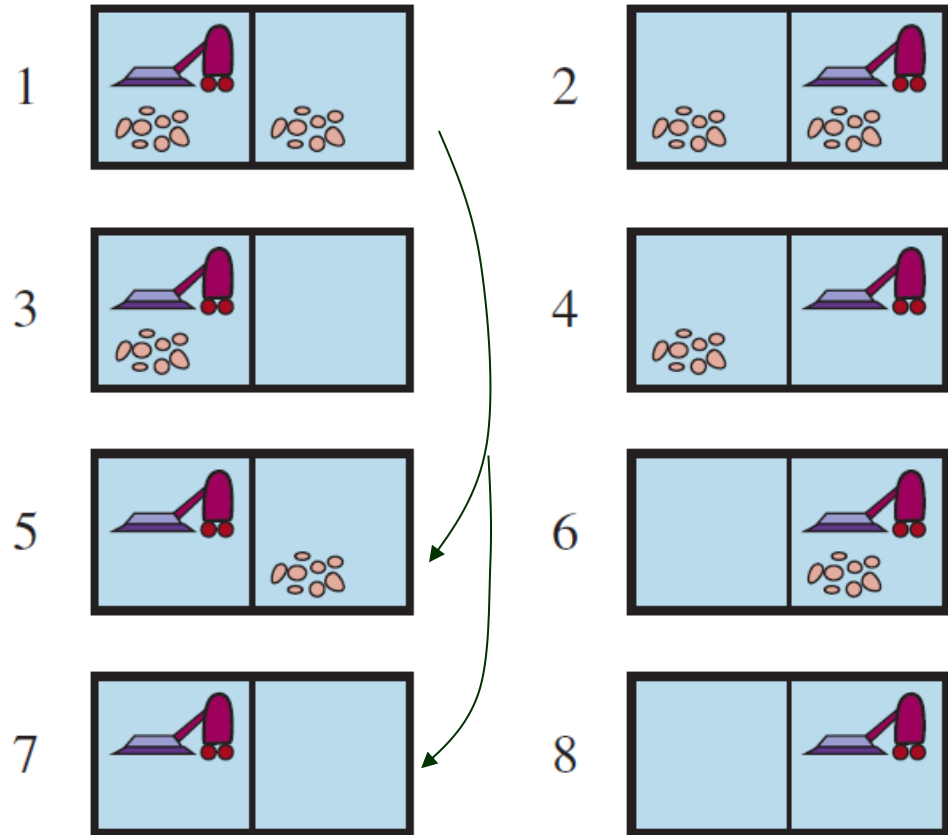
Erratic Vacuum World

Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square
- Applied to a clean square
 - ◆ sometimes deposit dirt on the carpet

Either state 5 or 7 after applied to state 1:

$$\text{RESULTS}(1, \text{Suck}) = \{5, 7\}$$



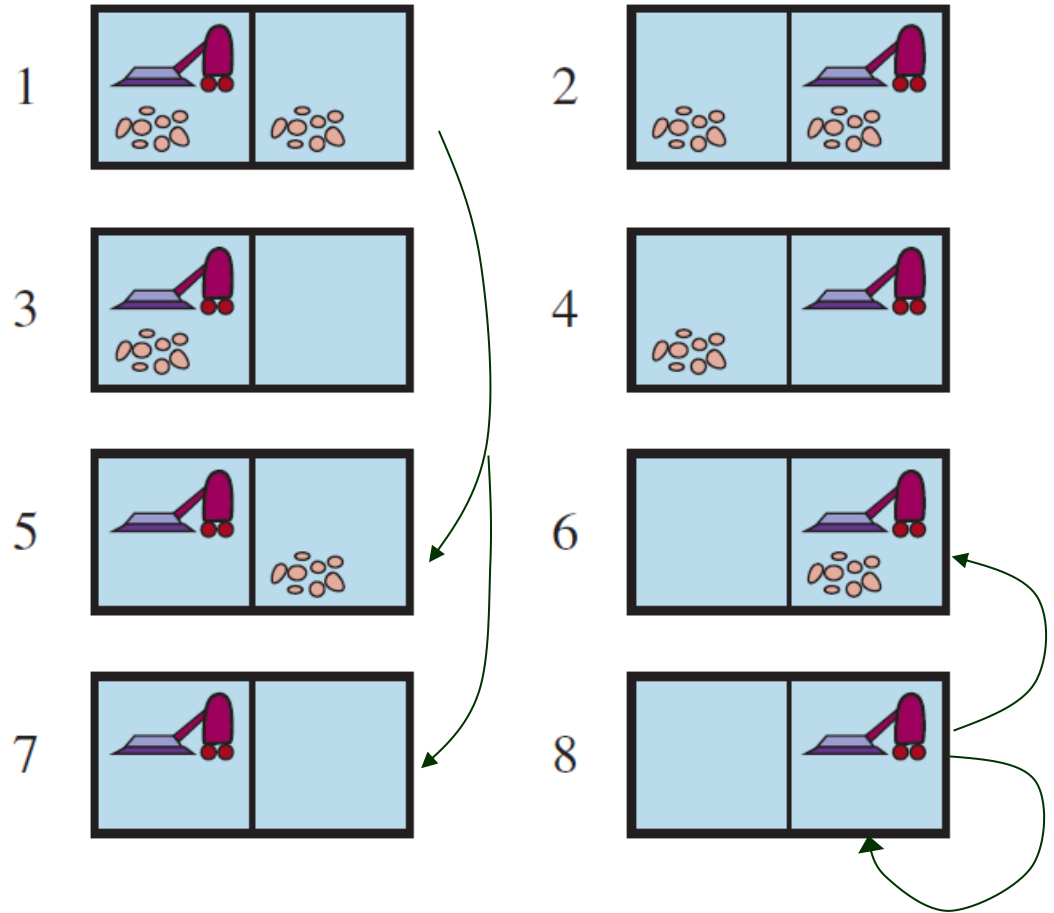
Erratic Vacuum World

Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square
- Applied to a clean square
 - ◆ sometimes deposit dirt on the carpet

Either state 5 or 7 after applied to state 1:

$\text{RESULTS}(1, \text{Suck}) = \{5, 7\}$



Erratic Vacuum World

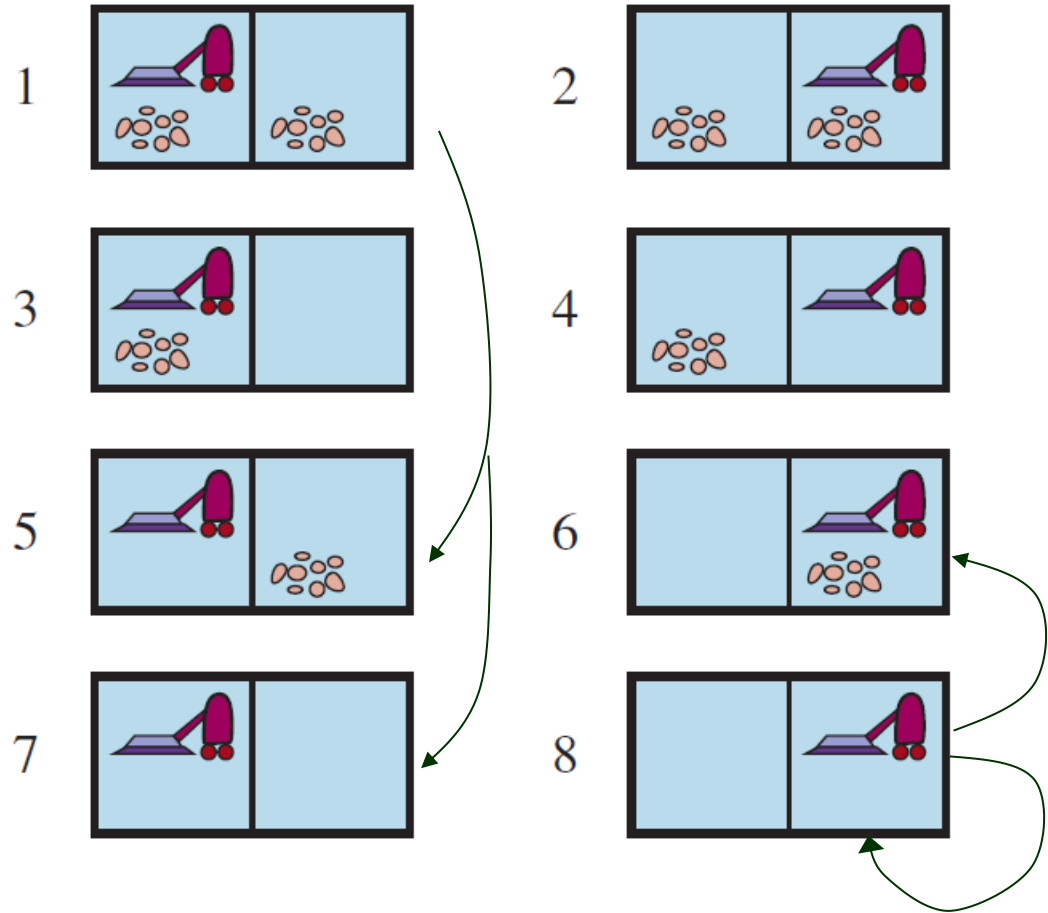
Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square
- Applied to a clean square
 - ◆ sometimes deposit dirt on the carpet

Either state 5 or 7 after applied to state 1:

$\text{RESULTS}(1, \text{Suck}) = \{5, 7\}$

$\text{RESULTS}(8, \text{Suck}) = \{6, 8\}$

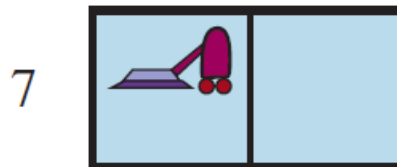
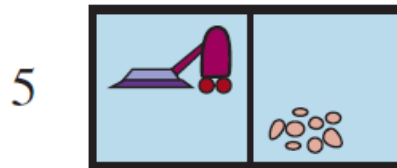
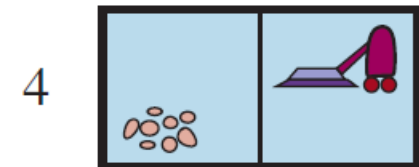
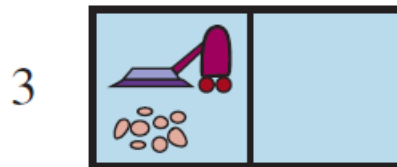
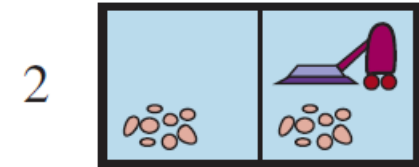
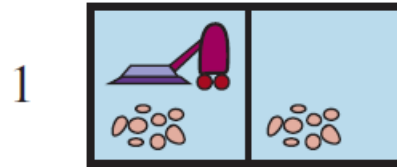


Still Solvable?

Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.



Still Solvable?

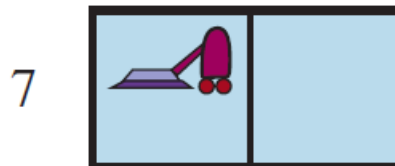
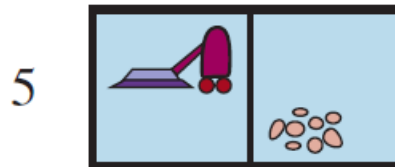
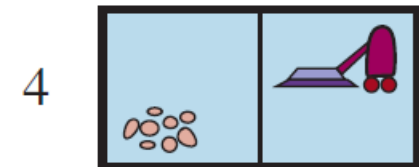
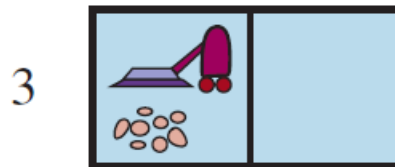
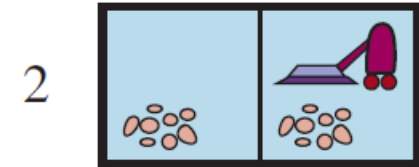
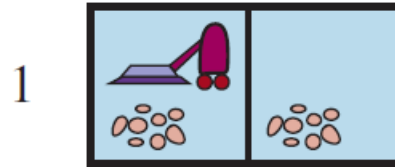
Suck:

- Applied to a dirty square
 - ♦ clean the square
 - ♦ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck*, if State = 5 then
[*Right*, *Suck*] else []]



Still Solvable?

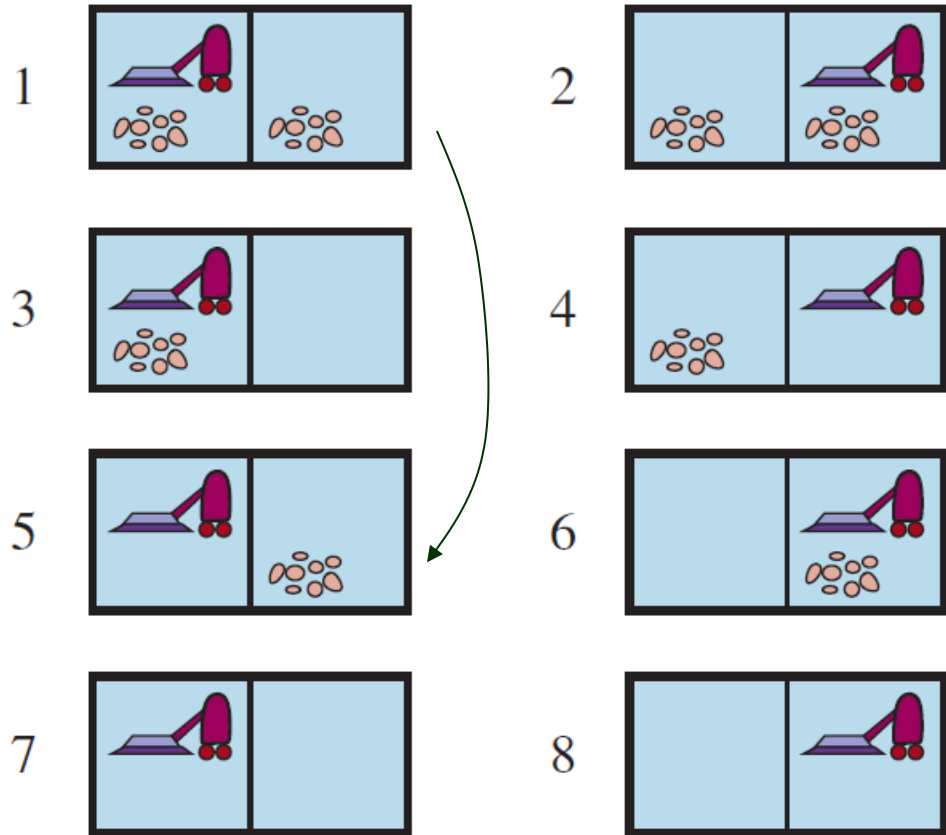
Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck*, if State = 5 then
[*Right*, *Suck*] else []]



Still Solvable?

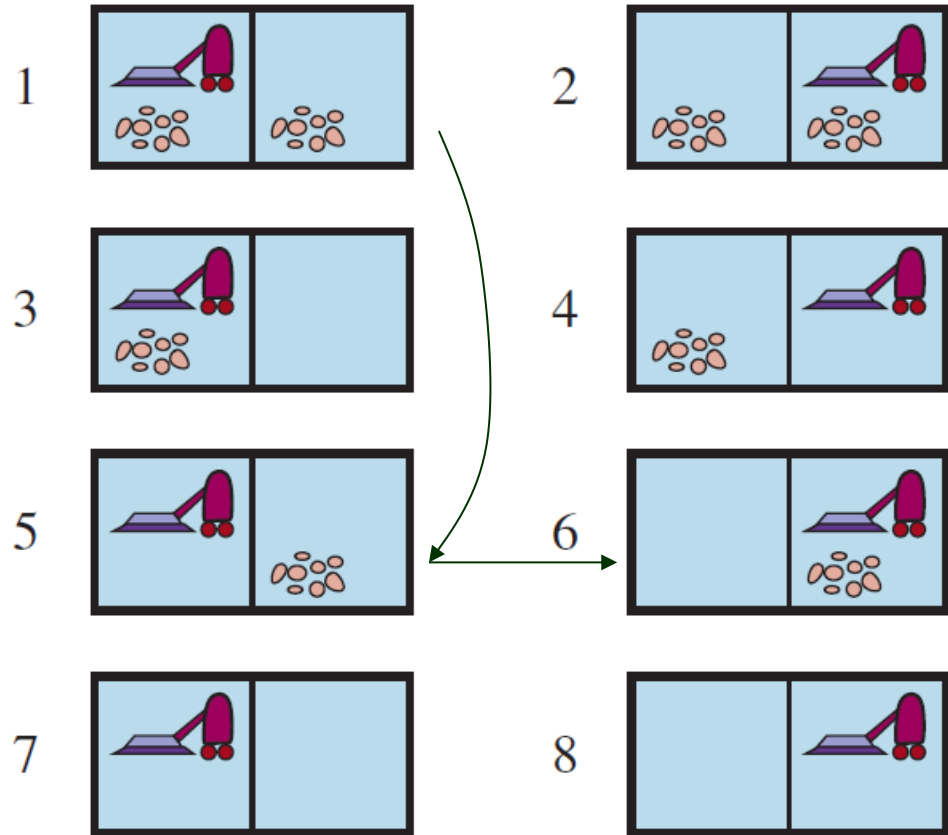
Suck:

- Applied to a dirty square
 - ♦ clean the square
 - ♦ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck*, if State = 5 then
[*Right*, *Suck*] else []]



Still Solvable?

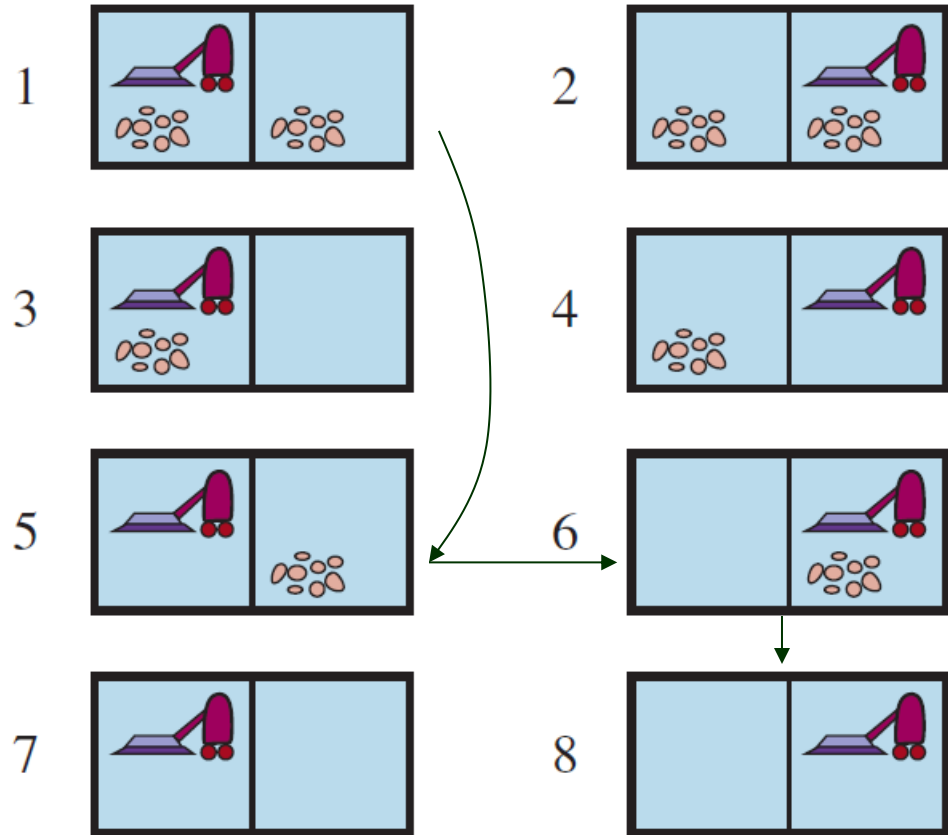
Suck:

- Applied to a dirty square
 - ♦ clean the square
 - ♦ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck*, if State = 5 then
[*Right*, *Suck*] else []]



Still Solvable?

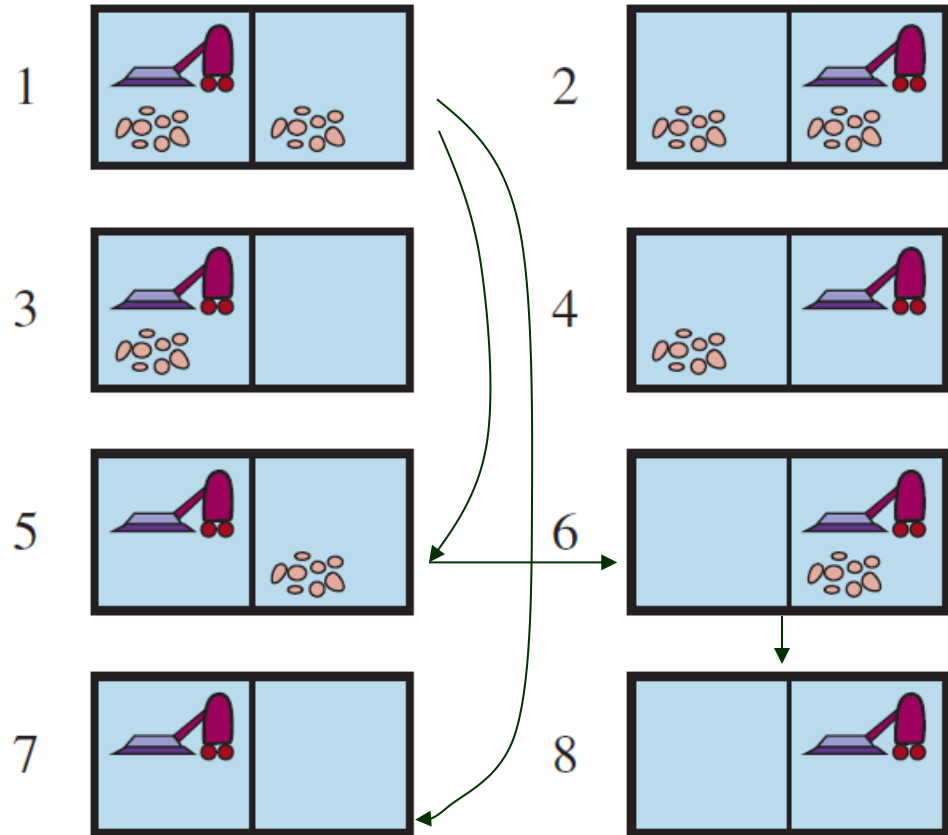
Suck:

- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[Suck, if State = 5 then
[Right, Suck] else []]



Still Solvable?

Suck:

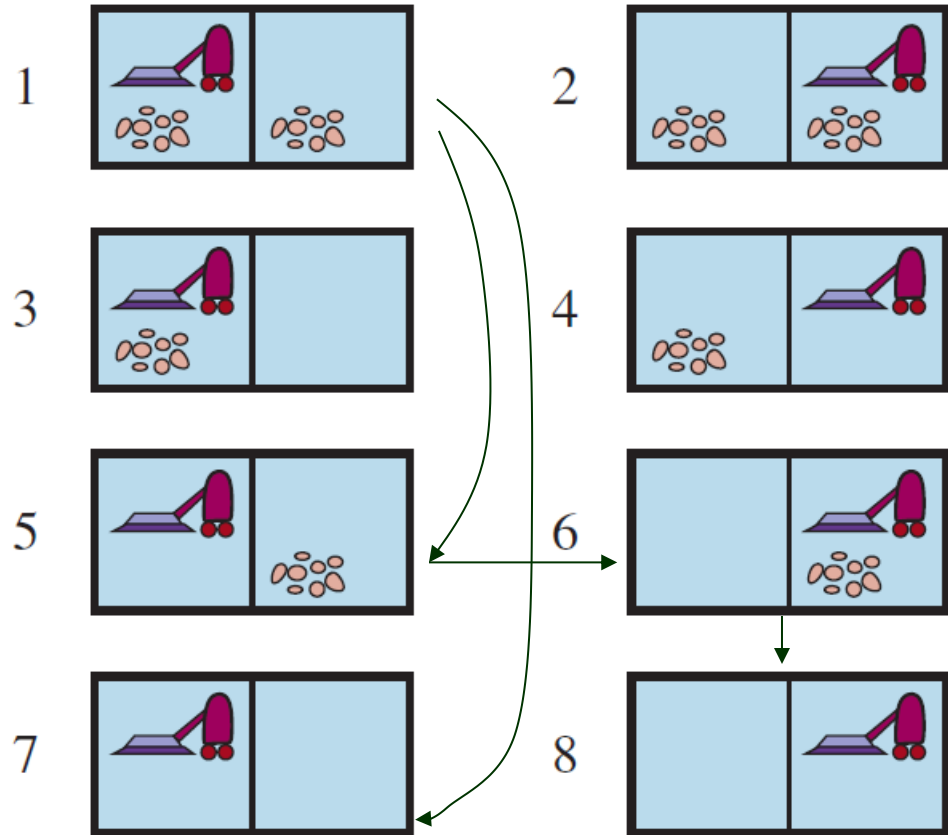
- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

No single sequence of actions solves the problem.

Solution still exists as a *conditional plan* (suppose at State 1):

[*Suck*, if State = 5 then
[*Right*, *Suck*] else []]

Solution is a tree – of a different character!



- Applied to a dirty square
 - ◆ clean the square
 - ◆ sometimes also clean up dirt in an adjacent square

- Applied to a clean square
 - ◆ sometimes deposit dirt on the carpet

OR-node (*deterministic*):
the agent chooses an action.

Suck:

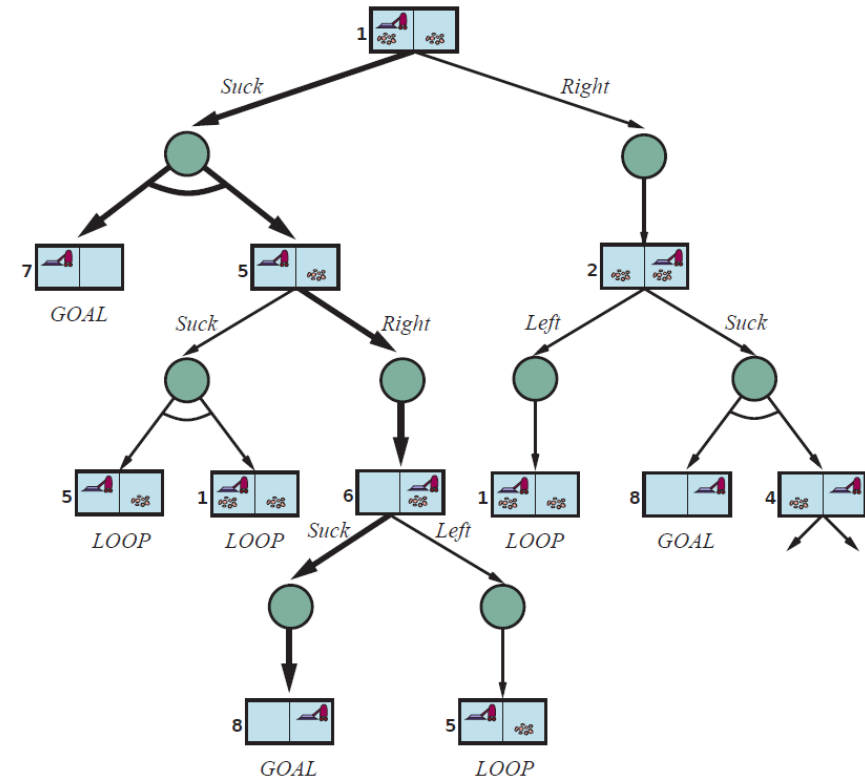
- Applied to a dirty square
 - ♦ clean the square
 - ♦ sometimes also clean up dirt in an adjacent square

AND-OR Search Tree

- Applied to a clean square
 - ♦ sometimes deposit dirt on the carpet

OR-node (*deterministic*):
the agent chooses an action.

e.g., *Left*, *Right*, or *Suck*



AND-node (*non-deterministic*): the environment “chooses” to have an outcome for each action.

e.g., *Suck* in state 1 results in the belief state {5, 7}.

Suck:

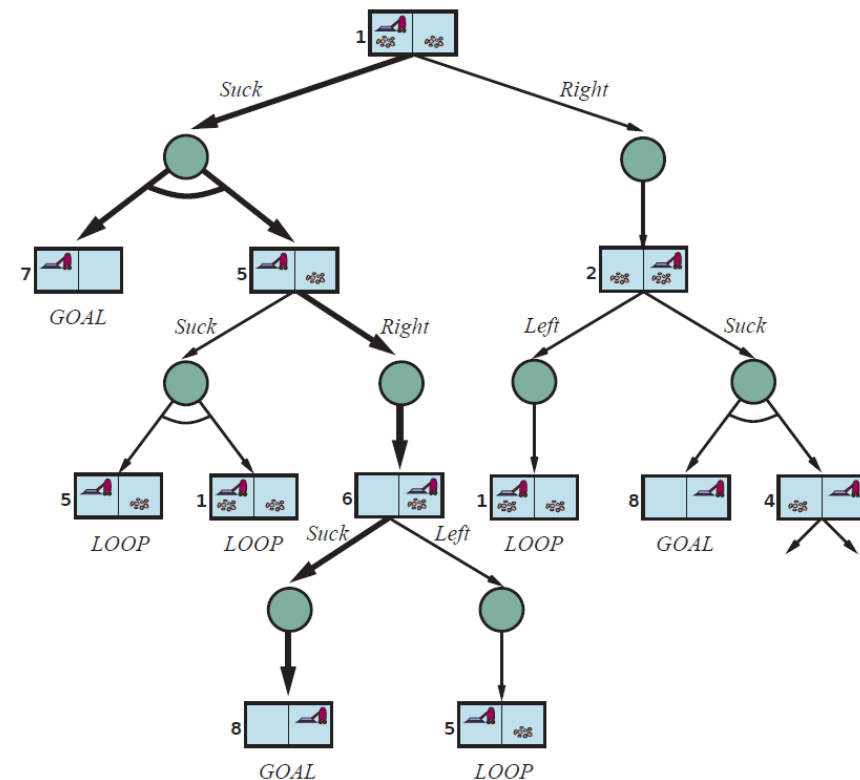
- Applied to a dirty square
 - ♦ clean the square
 - ♦ sometimes also clean up dirt in an adjacent square

AND-OR Search Tree

- Applied to a clean square
 - ♦ sometimes deposit dirt on the carpet

OR-node (*deterministic*):
the agent chooses an action.

e.g., *Left*, *Right*, or *Suck*

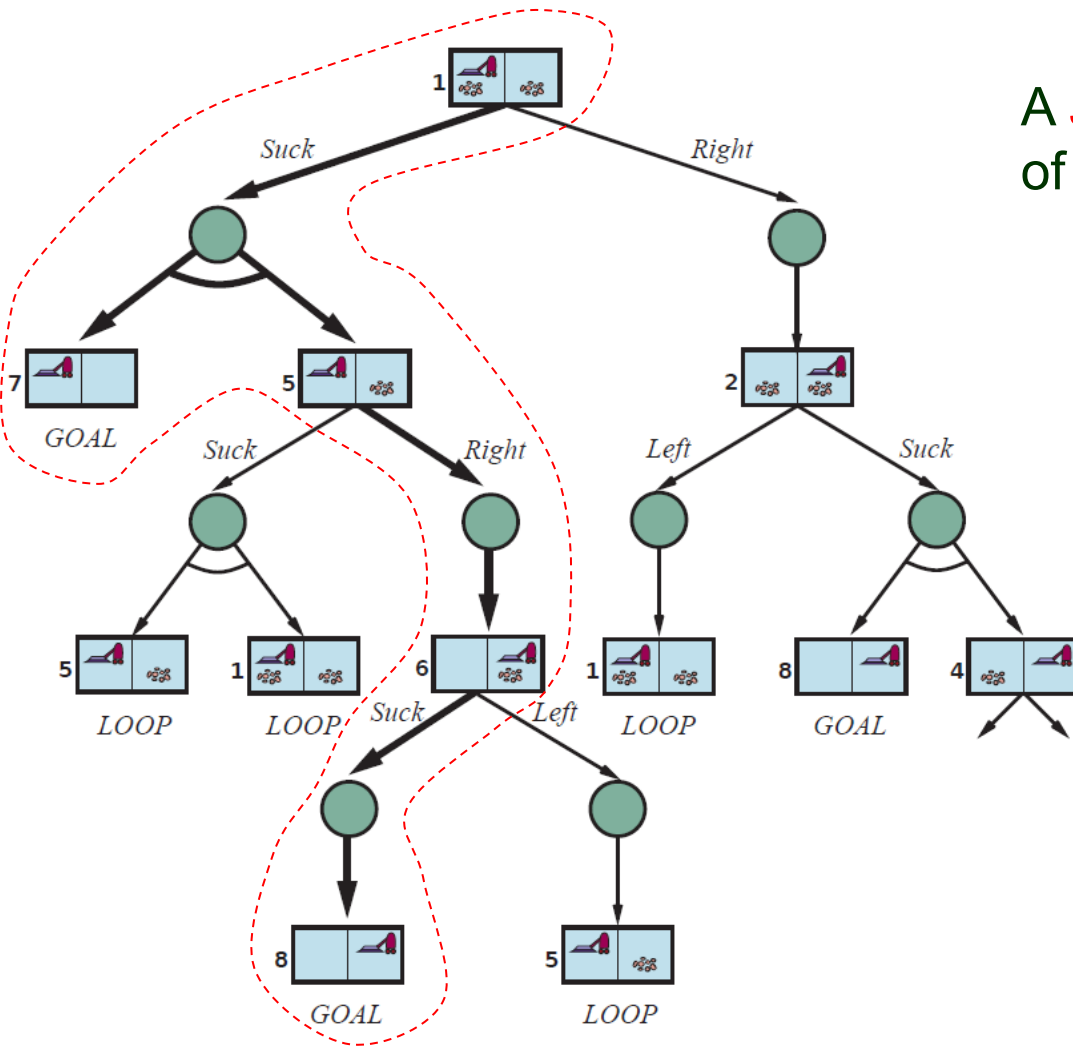


AND-node (*non-deterministic*): the environment “chooses” to have an outcome for each action.

e.g., *Suck* in state 1 results in the belief state {5, 7}.

OR- and AND-nodes alternate in the tree.

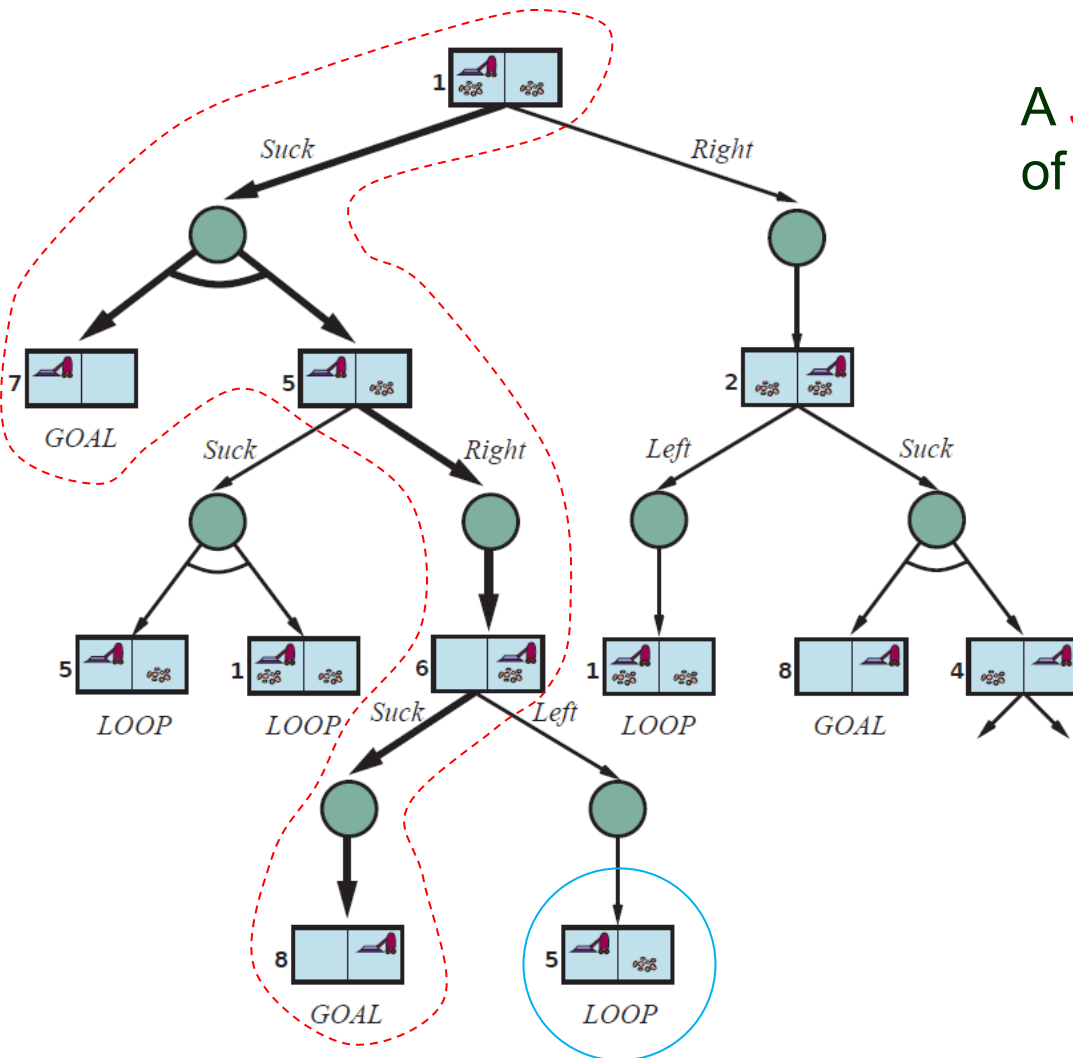
Example of a (Partial) Tree



A *solution* is a connected portion of the AND-OR tree such that

- ♦ its root is the tree's root;
- ♦ every OR node has exactly one child (i.e., one of the actions);
- ♦ every AND node has all children (possible outcomes) from the corresponding action;
- ♦ all the leaves are goal nodes.

Example of a (Partial) Tree



A *solution* is a connected portion of the AND-OR tree such that

- ♦ its root is the tree's root;
- ♦ every OR node has exactly one child (i.e., one of the actions);
- ♦ every AND node has all children (possible outcomes) from the corresponding action;
- ♦ all the leaves are goal nodes.

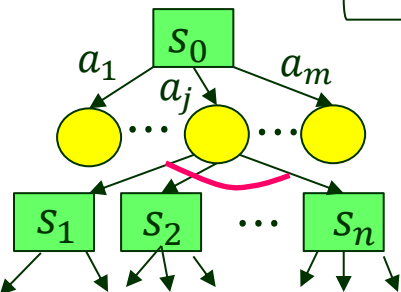
DFS Implementation of AND-OR Tree Search

function AND-OR-SEARCH(*problem*) **returns** a conditional plan, or *failure*
return OR-SEARCH(*problem*, *problem*.INITIAL, [])

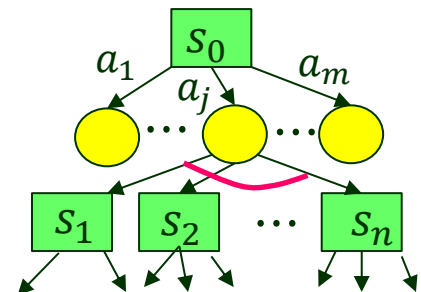
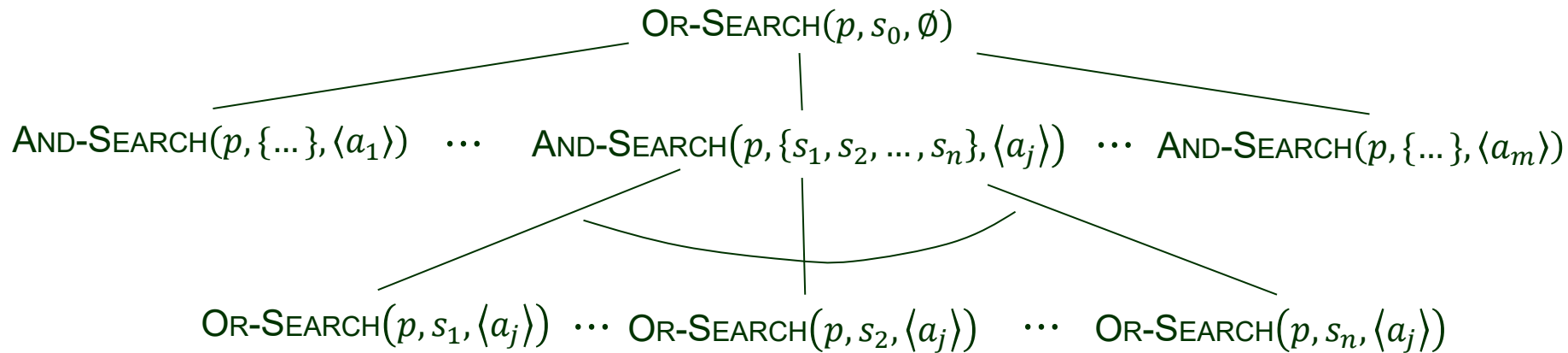
function OR-SEARCH(*problem*, *state*, *path*) **returns** a conditional plan, or *failure*
if *problem*.IS-GOAL(*state*) **then return** the empty plan
if IS-CYCLE(*path*) **then return** *failure* // ignore a solution with a cycle. such a solution would
for each *action* **in** *problem*.ACTIONS(*state*) **do** // imply the existence of a non-cyclic solution
 plan $\leftarrow$ AND-SEARCH(*problem*, RESULTS(*state*, *action*), [*state*] + *path*) // which can
 if *plan* $\neq$ *failure* **then return** [*action*] + *plan* // one action eventually leads
return *failure* // to a solution; terminates successfully.

function AND-SEARCH(*problem*, *states*, *path*) **returns** a conditional plan, or *failure*
for each *s_i* **in** *states* **do**
 plan_i $\leftarrow$ OR-SEARCH(*problem*, *s_i*, *path*)
 if *plan_i* = *failure* **then return** *failure* // one state has no solution; fails to solve the problem.
return [if *s₁* then *plan₁* else if *s₂* then *plan₂* else ... if *s_{n-1}* then *plan_{n-1}* else *plan_n*]

Solution plan



Recursion Tree

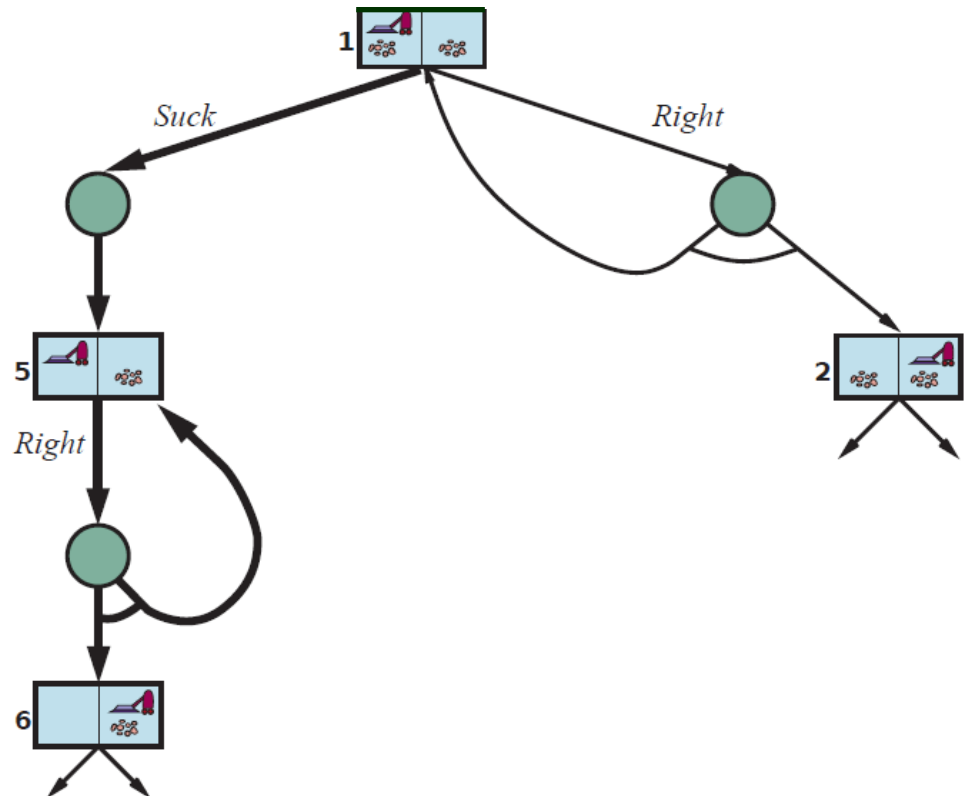


Cyclic Plan for a Solution

What if an action fails and the state is not changed?

Slippery vacuum world with deterministic Suck.

State 1 $\xrightarrow{\text{Right}}$ {1, 2}
State 5 $\xrightarrow{\text{Right}}$ {5, 6}



Cyclic Plan for a Solution

What if an action fails and the state is not changed?

Slippery vacuum world with deterministic Suck.

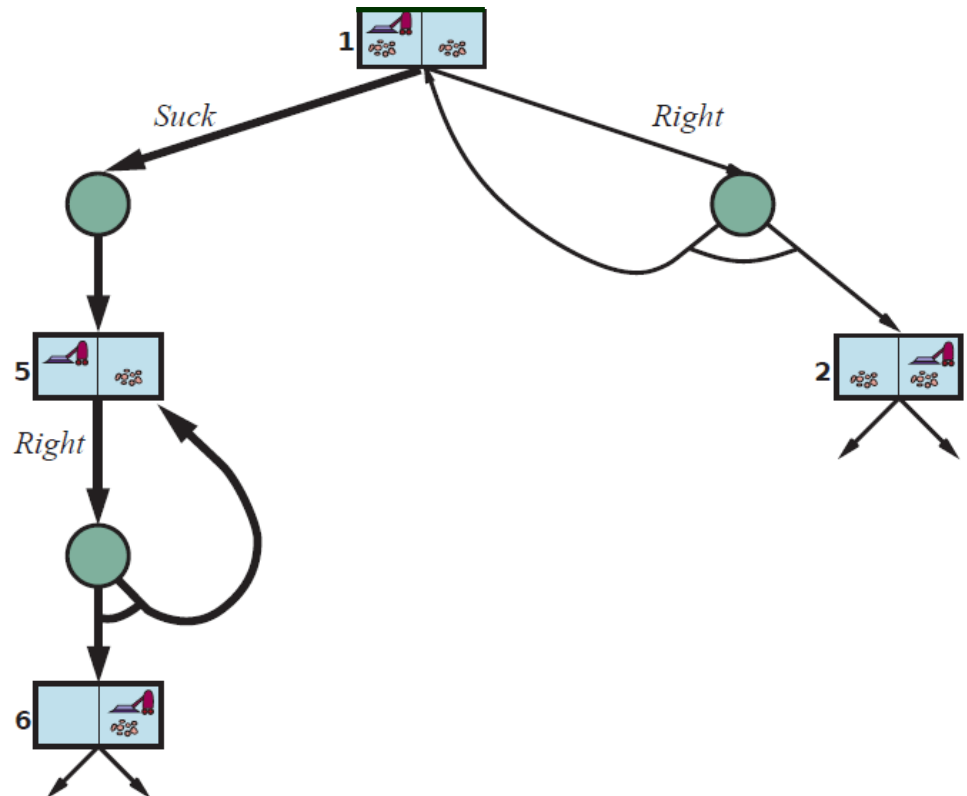
State 1 $\xrightarrow{\text{Right}}$ {1, 2}

State 5 $\xrightarrow{\text{Right}}$ {5, 6}

do

Suck;

if State = 5 then Right



Cyclic Plan for a Solution

What if an action fails and the state is not changed?

Slippery vacuum world with deterministic Suck.

State 1 $\xrightarrow{\text{Right}}$ {1, 2}

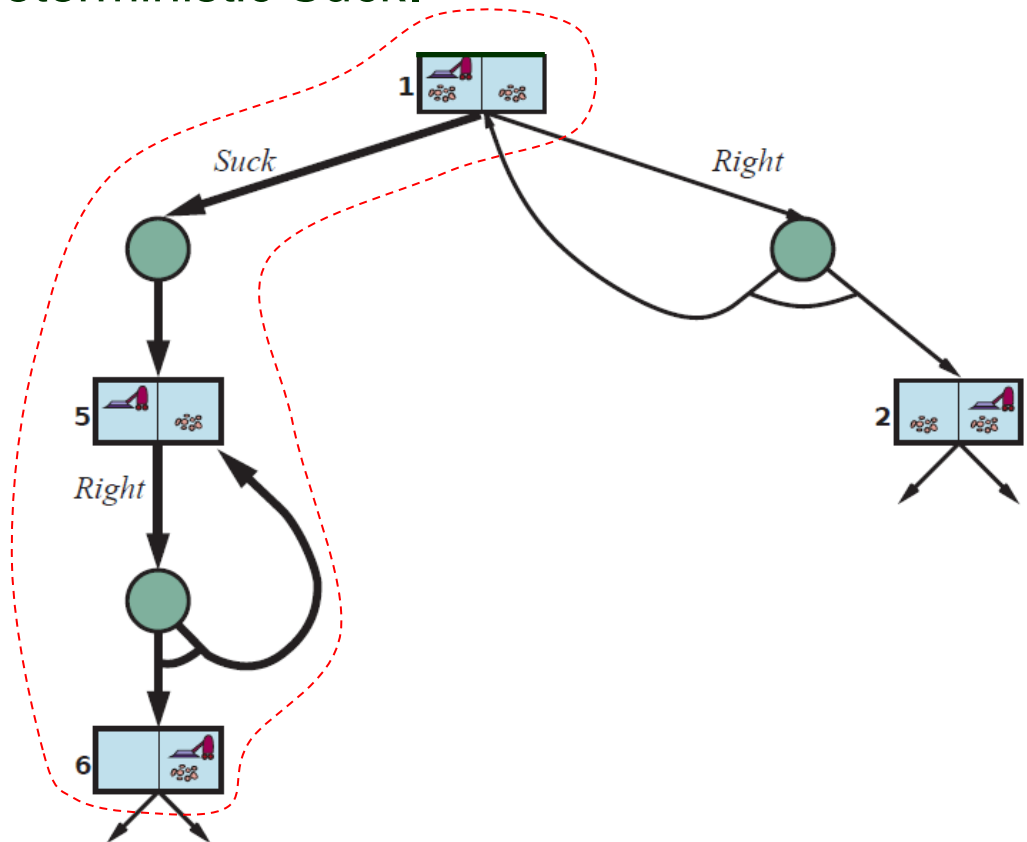
State 5 $\xrightarrow{\text{Right}}$ {5, 6}

Cyclic solution $\longrightarrow$

do

Suck;

if State = 5 then *Right*



Cyclic Plan for a Solution

What if an action fails and the state is not changed?

Slippery vacuum world with deterministic Suck.

State 1 $\xrightarrow{\text{Right}}$ {1, 2}

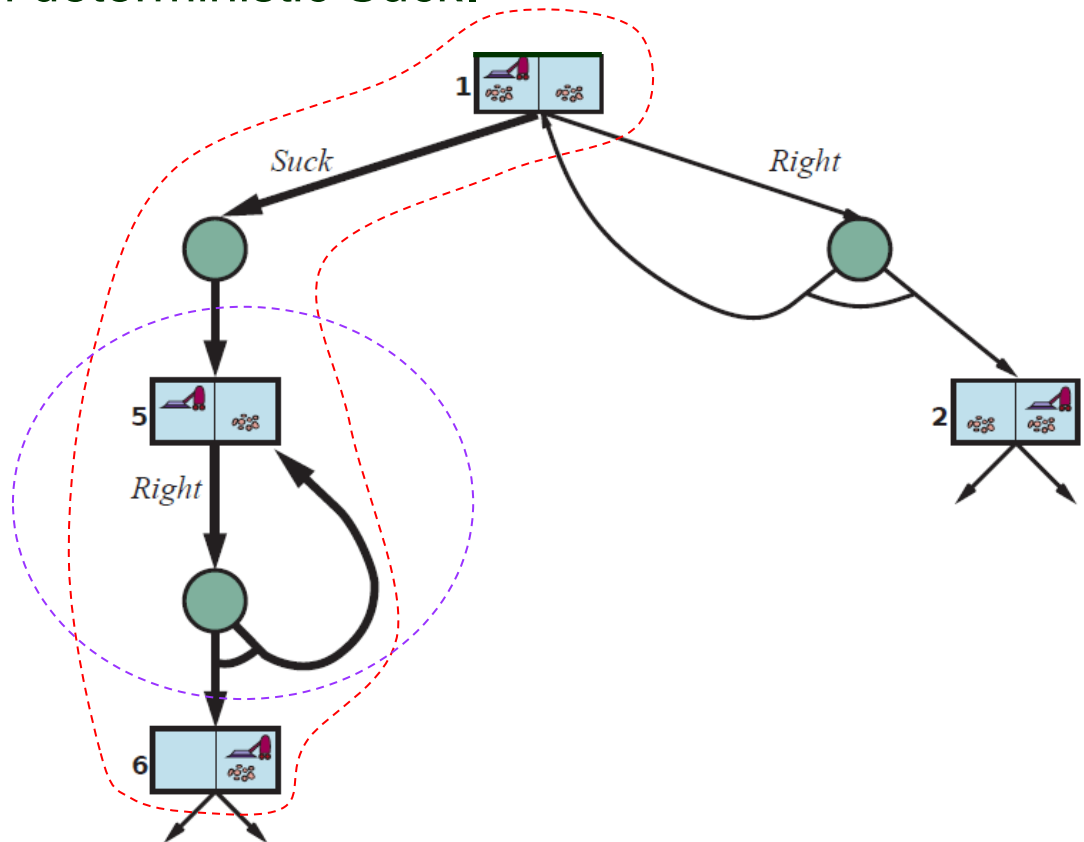
State 5 $\xrightarrow{\text{Right}}$ {5, 6}

Cyclic solution

do

Suck;

if State = 5 then Right



Cyclic Plan for a Solution

What if an action fails and the state is not changed?

Slippery vacuum world with deterministic Suck.

State 1 $\xrightarrow{\text{Right}}$ {1, 2}

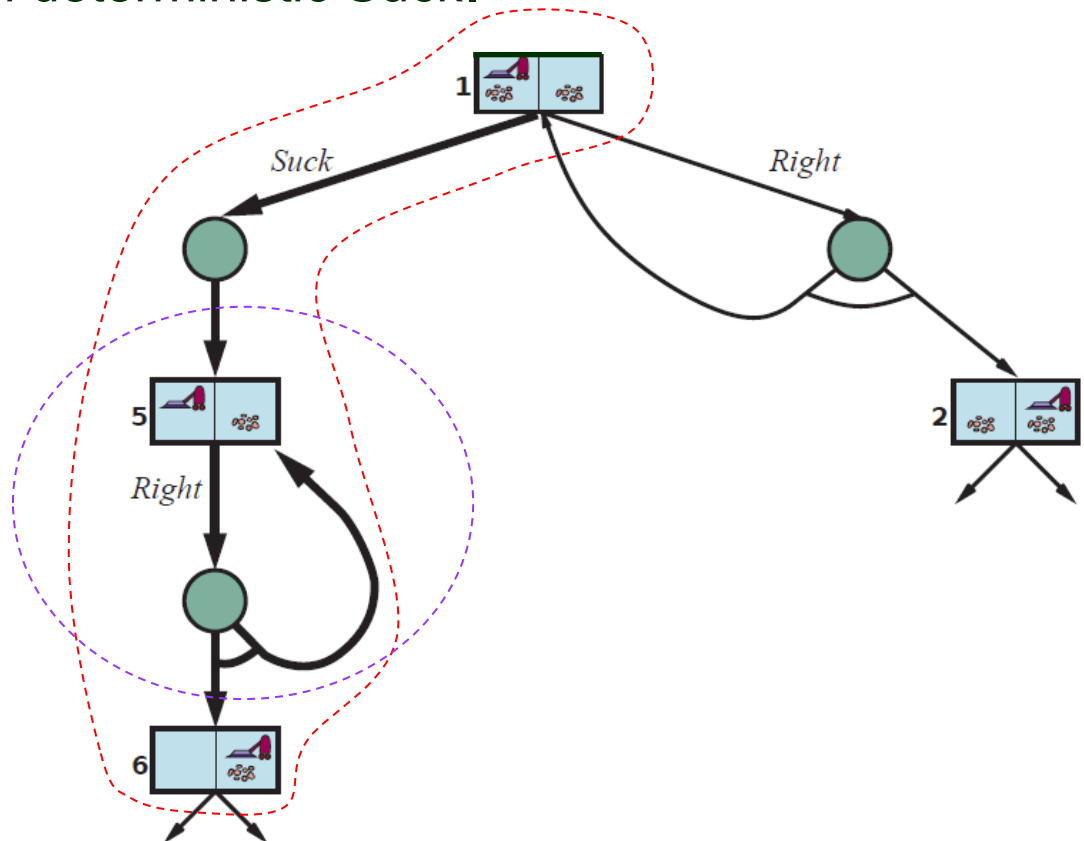
State 5 $\xrightarrow{\text{Right}}$ {5, 6}

Cyclic solution $\longrightarrow$

do

Suck;

if State = 5 then Right



The goal will be reached provided that each outcome of a nondeterministic action eventually occurs.